



UML Редактор

С генерацией кода

АННОТАЦИЯ

Книга посвящена проектированию и разработке LogicCraft — кроссплатформенного инструмента, автоматизирующего процесс создания программного обеспечения от построения UML-диаграмм до генерации готового кода на языках Python и Java. В издании подробно рассматривается архитектура приложения, построенная на паттерне MVC с применением графической библиотеки PyQt6.

ОБ АВТОРАХ

Важенин Александр Рудольфович — техник-программист

Болясников Семён Дмитриевич — инженер-программист

Беспрозванных Дарья Сергеевна — специалист по UX/UI

© Важенин А. Р., Болясников С. Д.,

Беспрозванных Д. С., 2026

© Оформление. Издательство, 2026

Оглавление

Введение.....	5
Глава 1. Подготовка к работе.....	5
1.1. Почему Python?.....	5
1.2. Используемые библиотеки.....	5
1.3 Обзор структуры проекта.....	5
Глава 2. Проектирование LogicCraft.....	6
2.1. Постановка задачи: UML-редактор с генерацией кода.....	6
2.2. Требования к функционалу	6
2.3. Выбор архитектурного подхода и технологий.....	6
2.4. Почему PyQt6 и MVC	6
2.5. Проектирование моделей данных UML	6
2.6. Разработка потоков работы пользователя.....	7
Глава 3. Архитектура приложения.....	7
3.1. Паттерн MVC в LogicCraft.....	7
3.2. Слои: Model, View, Controller, Services.....	8
3.3. Поток данных: от клика до результата	8
3.4. Связь компонентов через сигналы и слоты.....	9
Глава 4. Модели данных.....	10
4.1. UMLNode и UMLConnection	10
4.2. Типы элементов: класс, интерфейс, enum, абстрактный класс	11
4.3. Типы связей: ассоциация, наследование, композиция и другие	11
4.4. UMLDiagram — корневой объект проекта.....	12
4.5. Метаданные и версии диаграммы.....	13
4.6. Валидация данных с Pydantic.....	13
4.7. Сохранение и загрузка диаграммы в JSON.....	14
Глава 5. Графический интерфейс.....	16
5.1. QGraphicsScene — холст для диаграммы	16
5.2. UML-карточки: как нарисовать класс.....	17
5.3. Линии связей и стрелки	19
5.4. Точки привязки и создание связей мышью.....	23
5.5. Панель инструментов (Toolbox) и панель свойств (Properties).....	25
5.6. Система стилей: theme.py и style.qss	27
5.7. Масштабирование и панорамирование.....	28
5.8. Сетка и привязка к сетке	29
5.9. Выделение нескольких объектов.....	30
Глава 6. Контроллер и бизнес-логика.....	32
6.1. DiagramController — главный контроллер.....	32
6.2. Добавление, редактирование и удаление элементов	32

6.3. Паттерн Command и реализация Undo/Redo	34
6.4. Копирование, вставка, дублирование	36
6.5. История изменений	38
6.6. Валидация диаграммы и обработка ошибок	38
Глава 7. Генерация кода	40
7.1. Архитектура генератора кода	40
7.2. Промежуточная модель для генерации	43
7.3. Шаблоны Jinja2	43
7.4. Генерация Python-кода	44
7.5. Генерация Java-кода	46
7.6. Обработка наследования и связей	48
7.7. Генерация пакетов и модулей	48
7.8. Пользовательские шаблоны	49
7.9. Экспорт кода в файлы	50
Глава 8. Экспорт проекта	51
8.1. Структура готового проекта	51
8.2. Архитектурные шаблоны: MVC	52
8.3. Генерация конфигурационных файлов	52
8.4. Предпросмотр и экспорт на диск	54
Глава 9. Дополнительные UML-диаграммы	55
9.1. Use Case: актёры и сценарии	55
9.2. Связи Include и Extend	59
9.3. Переключение между типами диаграмм	60
9.4. Визуализация элементов разных диаграмм	63
Глава 10. Тестирование	69
10.1. Unit-тесты для моделей	69
10.2. Тесты генерации кода	69
10.3. Тестирование графического интерфейса	69
10.4. Запуск тестов через pytest	69
Глава 11. Сборка и развёртывание	69
11.1. Сборка исполняемого файла с PyInstaller	69
11.2. Запуск на Windows, macOS	69
Заключение	69

Введение

Современная разработка программного обеспечения всё чаще требует не только написания кода, но и его проектирования на более высоком уровне абстракции. Одним из таких подходов является использование UML-диаграмм, позволяющих визуально описывать структуру системы, её компоненты и взаимодействия.

LogicCraft — это приложение для проектирования UML-диаграмм с последующей генерацией исходного кода на различных языках программирования. Оно объединяет визуальное моделирование и автоматизацию разработки, позволяя сократить разрыв между архитектурой системы и её реализацией.

Основная цель проекта — предоставить удобный инструмент, в котором разработчик может:

- создавать UML-диаграммы в графическом интерфейсе;
- редактировать элементы системы (классы, интерфейсы, связи);
- автоматически генерировать код на популярных языках программирования;
- экспортировать готовую структуру проекта.

Приложение построено на архитектуре **MVC (Model-View-Controller)**, что обеспечивает чёткое разделение логики, интерфейса и управления данными. Для реализации используются современные технологии Python, включая **PyQt6** для графического интерфейса, **Pydantic** для валидации данных и **Jinja2** для генерации кода по шаблонам.

Таким образом, LogicCraft выступает не просто как UML-редактор, а как инструмент, объединяющий проектирование и разработку в единую рабочую среду.

Глава 1. Подготовка к работе

Перед началом работы с LogicCraft важно понять технологическую основу проекта, его структуру и используемые инструменты. Это позволяет быстрее ориентироваться в кодовой базе и понимать принципы работы приложения.

1.1. Почему Python?

Python выбран в качестве основного языка разработки LogicCraft благодаря сочетанию простоты и функциональности. Он позволяет быстро создавать сложные приложения, не перегружая код низкоуровневыми деталями.

Ключевые причины выбора:

- **Высокая скорость разработки** — Python позволяет быстрее реализовывать функциональность по сравнению с компилируемыми языками.
- **Читаемость и поддерживаемость** — код остаётся понятным даже при росте проекта.
- **Широкая экосистема** — доступ к библиотекам для GUI, работы с данными и генерации кода.
- **Кроссплатформенность** — одинаково работает на Windows, Linux и macOS.

1.2. Используемые библиотеки

В проекте LogicCraft применяются три ключевые библиотеки, каждая из которых отвечает за отдельный слой системы:

- **PyQt6** — используется для построения графического интерфейса приложения. На его основе реализованы редактор диаграмм, интерактивные элементы и обработка пользовательских действий.
- **Pydantic** — отвечает за описание и валидацию данных UML-моделей. Обеспечивает строгую структуру объектов и предотвращает появление некорректных данных в системе.
- **Jinja2** — используется для генерации исходного кода. Позволяет преобразовывать UML-модели в готовый код на разных языках программирования через шаблоны.

Эти инструменты вместе формируют основу приложения: интерфейс, данные и генерацию кода.

1.3 Обзор структуры проекта

LogicCraft организован модульно для удобства разработки и поддержки.

- **logiccraft/** — основной код: модели UML, графический интерфейс, контроллеры, сервисы.
- **templates/** — шаблоны Jinja2 для генерации кода на разных языках.
- **tests/** — автоматические тесты моделей, генератора кода и GUI.
- **docs/** — документация проекта.
- **pyproject.toml** — описание зависимостей и настроек проекта.

Такое построение позволяет легко ориентироваться в проекте и расширять функционал.

Глава 2. Проектирование LogicCraft

LogicCraft проектируется как UML-редактор с возможностью генерации исходного кода. Главная цель проекта — объединить визуальное моделирование системы и автоматическую генерацию программной реализации, чтобы ускорить процесс разработки и сделать его более наглядным.

2.1. Постановка задачи: UML-редактор с генерацией кода

Основная задача LogicCraft — предоставить разработчику инструмент, который позволяет:

- строить UML-диаграммы в интерактивном графическом интерфейсе;
- создавать и редактировать элементы диаграмм (классы, интерфейсы, enum, связи);
- сохранять диаграммы в структурированном формате для последующего редактирования;
- автоматически генерировать исходный код на нескольких языках программирования.

Таким образом, LogicCraft не просто рисует диаграммы, а превращает проектирование в готовый рабочий код, сокращая время разработки и снижая вероятность ошибок.

2.2. Требования к функционалу

Приложение должно обеспечивать:

- создание, редактирование и удаление UML-элементов;
- поддержку различных типов диаграмм (классов, связей, Use Case, Activity, Sequence);
- интуитивно понятный графический интерфейс с drag-and-drop и панелями свойств;
- сохранение и загрузку проектов в формате JSON;
- генерацию исходного кода на Python, Java, C#, JavaScript и других языках;
- расширяемость, позволяющую добавлять новые типы UML-элементов, связи и шаблоны кода.

Эти требования определяют ключевую ценность LogicCraft — сочетание удобного проектирования и автоматизации разработки.

2.3. Выбор архитектурного подхода и технологий

Для реализации LogicCraft был выбран Python и архитектура **MVC (Model-View-Controller)**.

Преимущества такого выбора:

- **Разделение логики и интерфейса** — модели данных не зависят от визуального представления, контроллер управляет связью между ними.
- **Масштабируемость** — легко добавлять новые функции и типы диаграмм.
- **Тестируемость** — отдельные компоненты можно проверять независимо.
- **Гибкость** — упрощается интеграция генератора кода и дополнительных модулей.

Python был выбран за его простоту, богатую экосистему библиотек и кроссплатформенность, что позволяет LogicCraft работать на Windows, macOS и Linux.

2.4. Почему PyQt6 и MVC

PyQt6 обеспечивает мощный инструмент для реализации графического интерфейса:

- система виджетов для создания панелей инструментов и свойств;
- графическая сцена (QGraphicsScene) для визуализации диаграмм и интерактивных элементов;
- удобная обработка событий, включая drag-and-drop, выделение и редактирование объектов.

Архитектура **MVC** позволяет разделить систему на три слоя:

- **Model** — данные UML: элементы, связи, диаграммы;
- **View** — визуальное представление элементов и сцены;
- **Controller** — управление действиями пользователя и обработка логики.

Это делает приложение структурированным, легко расширяемым и тестируемым.

2.5. Проектирование моделей данных UML

Основные модели LogicCraft:

- **UMLNode** — отдельный элемент диаграммы (класс, интерфейс, enum, абстрактный класс).
- **UMLConnection** — связи между элементами (ассоциация, наследование, композиция и др.).
- **UMLDiagram** — контейнер, объединяющий все элементы диаграммы и обеспечивающий их взаимодействие.

Модели проектируются так, чтобы быть независимыми от графического интерфейса и легко использоваться для генерации кода.

2.6. Разработка потоков работы пользователя

Типичный сценарий работы выглядит следующим образом:

1. Создание новой UML-диаграммы.
2. Добавление элементов: классы, интерфейсы, связи.
3. Настройка свойств элементов и их взаимного расположения.
4. Визуальная проверка структуры диаграммы.
5. Сохранение проекта в формате JSON для последующего редактирования.
6. Генерация исходного кода на выбранном языке программирования.

Этот поток отражает ключевую идею LogicCraft: от визуального проектирования к готовому коду с минимальными усилиями со стороны разработчика.

Глава 3. Архитектура приложения

3.1. Паттерн MVC в LogicCraft

В основе архитектуры приложения LogicCraft лежит классический паттерн проектирования Model-View-Controller (MVC). Выбор данного архитектурного подхода обусловлен необходимостью обеспечения чёткого разделения ответственности между компонентами системы, что критически важно при разработке сложных графических редакторов, таких как UML-инструменты. Разделение на слои позволяет упростить тестирование отдельных модулей, масштабировать функционал и обеспечить высокую степень расширяемости приложения без риска возникновения побочных эффектов в других частях кода.

Типичная структура MVC в LogicCraft:

src/logiccraft/

```
├── __main__.py # Точка входа
├── main.py # Инициализация приложения
├── models/ # Model - бизнес-логика и данные
├── views/ # View - пользовательский интерфейс
└── controllers/ # Controller - обработка действий
```

Реализация MVC в проекте организована по модульному принципу внутри каталога src/logiccraft/, где каждый слой имеет свою изолированную зону ответственности:

models/: Слой модели (Model) отвечает за представление данных и бизнес-логику. В контексте LogicCraft это классы, описывающие сущности UML-диаграммы: элементы (UMLNode), связи (UMLConnection) и сам контейнер диаграммы (UMLDiagram). Модель не содержит никакой информации о том, как данные отображаются пользователю; она лишь хранит состояние и правила его изменения.

views/: Слой представления (View) отвечает исключительно за визуализацию данных из модели и взаимодействие с пользователем через графический интерфейс. Реализация построена на базе библиотеки PyQt6, использующей компоненты QGraphicsScene и QGraphicsItem. Представление подписывается на изменения в модели и перерисовывает себя при их наступлении, но само не изменяет данные напрямую.

controllers/: Контроллерный слой (Controller) выступает в роли посредника между моделью и представлением. Он обрабатывает пользовательский ввод (клики мышью, ввод текста), преобразует эти действия в команды для слоя модели и инициирует обновление слоя представления. Например, контроллер получает сигнал от View о создании нового элемента, создаёт соответствующий объект в Model, а затем уведомляет View о необходимости отрисовать новый элемент на холсте.

Запуск приложения начинается с выполнения скрипта __main__.py, который вызывает функцию запуска из модуля main.py. Именно здесь происходит первичная настройка связей между компонентами MVC и запуск основного цикла обработки событий графического интерфейса.

Такая организация потоков данных обеспечивает предсказуемость поведения системы: любое изменение состояния проходит через единый центр управления — контроллер, что гарантирует целостность данных и упрощает отладку. Кроме того, строгая изоляция слоёв позволяет независимо разрабатывать и тестировать логику работы с данными, алгоритмы генерации кода и пользовательский интерфейс, используя такие инструменты, как pytest для юнит-тестирования моделей и GUI-тестов [sources=[file]].

3.2. Слои: Model, View, Controller, Services

Архитектура приложения реализует классическую схему MVC, дополненную сервисным слоем для инкапсуляции сложной бизнес-логики.

Model Layer (Слой моделей)

Расположение: `src/logiccraft/models/`.

Ответственность: хранение данных и бизнес-логика предметной области. Этот слой полностью независим от графического интерфейса.

Ключевые компоненты:

`diagram.py`: базовые модели данных (UMLDiagram, UMLNode, UMLConnection).

`diagram_manager.py`: CRUD-операции с диаграммой.

`engine.py`: валидация, статистика и анализ диаграммы.

`project_settings.py`: глобальные настройки проекта.

`structure_template.py`: шаблоны архитектурных структур.

View Layer (Слой представления)

Расположение: `src/logiccraft/view/`.

Ответственность: визуализация данных и взаимодействие с пользователем. Не содержит бизнес-логики.

Ключевые компоненты:

`main_window.py`: главное окно приложения (MainWindow) и виджет диаграммы (DiagramView).

`theme.py`: стилевые токены для унификации внешнего вида.

`scenes/diagram_scene.py`: графический холст (DiagramScene).

`widgets/`: кастомные UI-компоненты (UMLCard, ConnectionLine).

`panels/`: боковые панели (ToolboxPanel, PropertiesPanel).

`dialogs/`: диалоговые окна для редактирования.

Controller Layer (Слой контроллера)

Расположение: `src/logiccraft/controllers/`.

Ответственность: посредничество между Model и View. Обработывает пользовательский ввод, обновляет модель и инициирует перерисовку представления.

Ключевой компонент:

`diagram_controller.py`: главный контроллер, связывающий все части системы.

Механизм взаимодействия: использует сигналы (`card_added`, `diagram_loaded` и др.) для уведомления View об изменениях в модели и предоставляет API для управления диаграммой (`add_card()`, `remove_connection()`, `undo()`, `redo()`).

Service Layer (Слой сервисов)

Расположение: `src/logiccraft/services/`.

Ответственность: инкапсуляция сложной бизнес-логики и алгоритмов, используемых контроллером. Это позволяет контроллеру оставаться «тонким».

Ключевые компоненты:

`code_generator.py`: генерация исходного кода на различных языках.

`history_service.py`: реализация функционала Undo/Redo.

`project_exporter.py`: экспорт структуры проекта.

`serialization_service.py`: сериализация и десериализация диаграммы.

`geometry_service.py`: геометрические вычисления для отрисовки.

3.3. Поток данных: от клика до результата

Взаимодействие пользователя с LogicCraft представляет собой строго определённый поток данных, который проходит через все архитектурные слои: от графического интерфейса до обновления модели и обратно. Этот механизм обеспечивает целостность данных и предсказуемость поведения системы.

Рассмотрим ключевые сценарии.

Создание карточки (класса)

Пользователь нажимает кнопку «Add Class» в панели инструментов.

MainWindow обрабатывает клик и генерирует сигнал `add_card_requested` с координатами.

DiagramController получает сигнал и вызывает метод `add_card(x, y)`.

Контроллер создаёт объект UMLNode в модели и генерирует сигнал `card_added(node)`.

MainWindow, получив сигнал, создаёт визуальный компонент UMLCard и добавляет его на графическую сцену.

Создание связи

Пользователь нажимает на точку привязки (AnchorPoint) одного элемента и тянет курсор к другому.

DiagramScene создаёт временную пунктирную линию (temp_line) для визуализации процесса.

При отпускании мыши над точкой привязки другого элемента сцена генерирует сигнал connection_ready с идентификаторами источника и цели.

DiagramController получает сигнал, создаёт объект UMLConnection в модели и генерирует сигнал connection_added(conn).

На основе этого сигнала создаётся постоянный визуальный компонент ConnectionLine, который добавляется на сцену.

Перемещение элемента

Пользователь перетаскивает карточку UMLCard.

Карточка генерирует внутренний сигнал об изменении позиции, на который подписан объект ConnectionLine. Это приводит к немедленному пересчёту геометрии линии связи.

По завершении перемещения карточка генерирует сигнал card_moved(card_id, x, y).

DiagramController получает этот сигнал и обновляет координаты соответствующего узла в модели.

Отмена действия (Undo/Redo)

Пользователь нажимает Ctrl+Z.

DiagramController вызывает метод undo() у сервиса истории (HistoryService).

Сервис восстанавливает предыдущее состояние модели (UMLDiagram) и возвращает его контроллеру.

Контроллер обновляет свою внутреннюю ссылку на модель и генерирует сигнал diagram_loaded.

MainWindow очищает графическую сцену и полностью перестраивает её визуальное представление на основе нового состояния модели.

Этот однонаправленный

поток данных — от View

к Controller,

затем к Model

(и, при необходимости, к Services)

и обратно к View

через сигналы — является ядром логики приложения, гарантируя, что визуальное состояние всегда синхронизировано с данными в модели.

3.4. Связь компонентов через сигналы и слоты

Взаимодействие между компонентами LogicCraft построено на механизме сигналов и слотов, что обеспечивает слабую связанность (loose coupling) между слоями. Компоненты не вызывают методы друг у друга напрямую, а обмениваются сообщениями о событиях, на которые другие могут подписываться.

Связи можно разделить на

несколько ключевых направлений.

1. От View к Controller

Это основной поток,

инициируемый действиями пользователя в интерфейсе. Представление генерирует

сигналы, а контроллер или промежуточное приложение обрабатывает их.

Сигналы от MainWindow:

add_card_requested(x, y, node_type) → Application._on_add_card()

save_requested(filepath) → Application._on_save()

load_requested(filepath) → Application._on_load()

clear_requested() → Application._on_clear()

edit_card_requested(card_id, ...) → Application._on_edit_card()

delete_selected_requested() → Application._on_delete_selected()

Сигналы от DiagramScene (графическая сцена):

connection_ready(source_id, target_id, ...) → MainWindow._on_connection_ready()

card_moved(card_id, x, y) → MainWindow._on_card_moved()

2. От Controller к View

Этот поток используется

для уведомления представления об изменениях в модели, чтобы оно могло обновить

свой вид.

Сигналы от DiagramController:

card_added(node) → Application._on_card_added()

card_removed(card_id) → Application._on_card_removed()

connection_added(connection) → Application._on_connection_added()

diagram_cleared() → Application._on_diagram_cleared()

diagram_loaded() → Application._on_diagram_loaded()

status_changed(message) → MainWindow.update_status()

3. Внутренние сигналы View и от виджетов

Эти сигналы используются

для координации элементов внутри графического интерфейса.

Сигналы от UMLCard:

position_changed() (подписчик — ConnectionLine.update_position()): для мгновенного обновления линии связи при перемещении карточки.

move_finished(card_id, x, y) → Application._on_card_move_finished(): для фиксации окончательной позиции в модели.

4. От панелей инструментов и свойств

Toolbox Panel (панель инструментов) → View:

add_element_requested(node_type) → MainWindow._on_toolbox_add_element()

Properties Panel (панель свойств) → Controller:

name_changed(card_id, name) → controller.edit_card()

attributes_changed(card_id, attributes) → controller.edit_card()

5. От сервисных слоев

History Service (сервис истории) → Controller:

state_restored(diagram) → DiagramController._on_state_restored(): уведомляет контроллер о восстановлении состояния для выполнения Undo/Redo.

Все эти соединения

централизованно настраиваются в точке входа приложения (main.py), где происходит связывание сигналов

и соответствующих слотов-обработчиков.

Глава 4. Модели данных

Слой моделей (Model) в LogicCraft абстрагирован от графического интерфейса. Это изолированное хранилище состояния системы, гарантирующее валидность структуры диаграммы на любом этапе её жизненного цикла — от редактирования до генерации кода.

4.1. UMLNode и UMLConnection

Ядром объектной модели являются две независимые сущности: узлы и связи.

- **UMLNode** — представляет собой абстрактное описание любого элемента диаграммы. Помимо текстовых полей (имя, свойства, методы), узел содержит пространственные координаты (x, y). Это сознательное отступление от «чистого» MVC в пользу минимизации накладных расходов: хранение координат непосредственно в модели упрощает сериализацию всей сцены в JSON.

- **UMLConnection** — описывает топологию графа диаграммы. Модель связи оперирует исключительно строковыми идентификаторами **source_id** и **target_id**. Она также фиксирует конкретные точки привязки (анкеры) на сторонах карточек, что критически важно для корректного пересчёта геометрии стрелок при перемещении узлов.

class UMLConnection(BaseModel):

id: str = Field(default_factory=lambda: str(uuid4()))

source_id: str

target_id: str

type: ConnectionType = ConnectionType.association

source_anchor: str = "right"

target_anchor: str = "left"

multiplicity: Optional[str] = None

name: Optional[str] = None

4.2. Типы элементов: класс, интерфейс, enum, абстрактный класс

Для разграничения бизнес-логики и последующей генерации кода каждый объект `UMLNode` строго типизируется с помощью перечисления `NodeType`. Каждому типу соответствуют свои правила отображения и генерации:

```
from pydantic import BaseModel, Field
from enum import Enum
from uuid import uuid4
from typing import Optional, List
```

```
class NodeType(str, Enum):
    CLASS = "class"
    INTERFACE = "interface"
    ENUM = "enum"
    ABSTRACT_CLASS = "abstract_class"
```

```
class DiagramType(str, Enum):
    CLASS = "class"
    USE_CASE = "use_case"
```

```
class UMLEnumLiteral(BaseModel):
    name: str
    value: Optional[str] = None
```

```
class UMLProperty(BaseModel):
    name: str
    type: str
    visibility: str = "public"
    is_static: bool = False
    default_value: Optional[str] = None
```

```
class UMLMethod(BaseModel):
    name: str
    return_type: Optional[str] = "void"
    visibility: str = "public"
    is_abstract: bool = False
    is_static: bool = False
    parameters: list[UMLProperty] = Field(default_factory=list)
```

4.3. Типы связей: ассоциация, наследование, композиция и другие

Тип связи (`ConnectionType`) в модели `UMLConnection` является главным триггером для построения архитектурных отношений в результирующих файлах кода:

- **INHERITANCE (Наследование) / IMPLEMENTATION (Реализация):** Указывает генератору кода на необходимость добавить целевой класс в список базовых классов (родителей) для класса-источника.
 - В Python: `class Source(Target):`
 - В Java: `class Source extends Target {` или `implements Target.`
- **COMPOSITION (Композиция) / AGGREGATION (Агрегация):** Сигнализирует о том, что класс-источник должен содержать в своих свойствах поле, ссылающееся на экземпляр целевого класса (или список экземпляров, если множественность равна `*`). В случае композиции в конструктор источника может быть заложена логика жесткого жизненного цикла целевого объекта.
- **ASSOCIATION (Ассоциация) / DEPENDENCY (Зависимость):** Означает более слабые типы связей (например, использование целевого класса в качестве аргумента метода или временной локальной переменной). В генераторе кода это может выражаться в автоматическом добавлении директивы `import` в начале файла.

```
class ConnectionType(str, Enum):
    association = "association" # простая связь
    inheritance = "inheritance" # наследование
    composition = "composition" # композиция
    aggregation = "aggregation" # агрегация
    dependency = "dependency" # зависимость
    interaction = "interaction" # взаимодействие
```

```

realization = "realization" # реализация интерфейса
# Use Case связи
uc_association = "uc_association" # актёр — сценарий
uc_include = "uc_include" # <<include>>
uc_extend = "uc_extend" # <<extend>>

```

```

class UMLConnection(BaseModel):
    id: str = Field(default_factory=lambda: str(uuid4()))
    source_id: str
    target_id: str
    type: ConnectionType = ConnectionType.association
    source_anchor: str = "right"
    target_anchor: str = "left"
    multiplicity: Optional[str] = None
    name: Optional[str] = None

```

4.4. UMLDiagram — корневой объект проекта

Для управления всей совокупностью элементов нам необходим единый контейнер, который будет агрегировать списки узлов и связей, а также предоставлять верхнеуровневый доступ к ним. Модель UMLDiagram является "корневым агрегатом" (Aggregate Root) в терминах предметно-ориентированного проектирования (DDD).

```

class DiagramType(str, Enum):
    CLASS = "class"
    USE_CASE = "use_case"

class UMLDiagram(BaseModel):
    id: str = Field(default_factory=lambda: str(uuid4()))
    name: str
    diagram_type: DiagramType = DiagramType.CLASS
    nodes: list[UMLNode] = Field(default_factory=list)
    connections: list[UMLConnection] = Field(default_factory=list)
    uc_actors: List["UseCaseActor"] = Field(default_factory=list)
    uc_scenarios: List["UseCaseScenario"] = Field(default_factory=list)
    uc_connections: List["UseCaseConnection"] = Field(default_factory=list)

    def get_node(self, node_id: str) -> Optional[UMLNode]:
        for node in self.nodes:
            if node.id == node_id:
                return node
        return None

    def get_connections_for_node(self, node_id: str) -> list[UMLConnection]:
        return [conn for conn in self.connections
                if conn.source_id == node_id or conn.target_id == node_id]

    def validate(self) -> list[str]:
        errors = []
        node_ids = {node.id for node in self.nodes}

        for conn in self.connections:
            if conn.source_id not in node_ids:
                errors.append(f"Connection {conn.id}: source {conn.source_id} not found")
            if conn.target_id not in node_ids:
                errors.append(f"Connection {conn.id}: target {conn.target_id} not found")

        return errors

```

Благодаря такой структуре, вся диаграмма представляет собой единый строго типизированный объект, который можно легко скопировать целиком или передать в сервис сохранения.

4.5. Метаданные и версии диаграммы

Обратите внимание на поля `project_name` и `version` в корневой модели диаграммы. Логирование версий схемы данных проекта критически важно при развитии программного продукта.

Если в будущих версиях LogicCraft модель `UMLNode` расширится новыми обязательными полями (например, добавятся параметры для стилизации конкретной карточки), то файлы проектов, созданные пользователями в старых версиях программы, перестанут открываться из-за ошибок валидации. Наличие метаданных о версии проекта позволяет слою сериализации своевременно перехватывать старые структуры данных и прогонять их через функции-миграции перед парсингом.

4.6. Валидация данных с Pydantic

Pydantic выполняет проверку типов автоматически в момент инициализации объектов или присвоения им значений (если включена соответствующая настройка).

`src/logiccraft/models/diagram.py` — метод `validate()` : проверяет, что все связи (`connections`) ссылаются на существующие узлы.

```
def validate(self) -> list[str]:
    errors = []
    node_ids = {node.id for node in self.nodes}

    for conn in self.connections:
        if conn.source_id not in node_ids:
            errors.append(f"Connection {conn.id}: source {conn.source_id} not found")
        if conn.target_id not in node_ids:
            errors.append(f"Connection {conn.id}: target {conn.target_id} not found")
    return errors
```

`src/logiccraft/services/history_service.py` — метод `validate_for_codegen()` проверяет битые связи и циклы наследования (алгоритм «Черепаша и Заяц»).

```
def validate_for_codegen(self, diagram: UMLDiagram) -> List[str]:
    errors = []
    node_ids = {n.id for n in diagram.nodes}

    for conn in diagram.connections:
        if conn.source_id not in node_ids or conn.target_id not in node_ids:
            errors.append(f"Broken connection detected: {conn.id}")

    inheritance_map = {
        c.source_id: c.target_id
        for c in diagram.connections if c.type.value == "inheritance"
    }

    for start_id in inheritance_map:
        slow = start_id
        fast = start_id
        while fast in inheritance_map and inheritance_map[fast] in inheritance_map:
            slow = inheritance_map[slow]
            fast = inheritance_map[inheritance_map[fast]]
        if slow == fast:
            node = diagram.get_node(slow)
            msg = f"Cycle detected in inheritance starting from: {node.name if node else slow}"
            errors.append(msg)
            self.validation_failed.emit(msg)
            break
```

```
return errors
```

src/logiccraft/controllers/diagram_controller.py — метод `validate_diagram()` : проверяет дублирующиеся имена классов и классы без атрибутов/методов.

```
def validate_diagram(self) -> list:
    warnings = []
    diagram = self.manager.diagram

    names = [n.name for n in diagram.nodes]
    duplicates = {n for n in names if names.count(n) > 1}
    for name in duplicates:
        warnings.append(f"⚠ Дублирующееся имя класса: «{name}»")

    for node in diagram.nodes:
        if not node.properties and not node.methods and node.node_type.value == 'class':
            warnings.append(f"❗ Класс «{node.name}» не имеет атрибутов и методов")

    for node in diagram.nodes:
        if node.node_type.value == 'interface' and not node.methods:
            warnings.append(f"⚠ Интерфейс «{node.name}» не имеет методов")
```

Если контроллер попытается записать в модель класса пустое имя или добавить метод в перечисление, Pydantic выбросит исключение `ValidationError`. Контроллер перехватит его и выведет пользователю понятное предупреждение в UI, защитив внутреннее состояние данных от разрушения.

4.7. Сохранение и загрузка диаграммы в JSON

Одним из главных преимуществ использования Pydantic является простота перевода моделей в текстовый формат и обратно. Нам больше не нужно вручную писать парсеры словарей.

src/logiccraft/models/diagram_manager.py — метод `save_to_file()` : сохраняет диаграмму в JSON через `json.dump()`, сериализуя узлы, связи, актёров и сценарии Use Case.

```
def save_to_file(self, filepath: str) -> bool:
    try:
        directory = Path(filepath).parent
        directory.mkdir(parents=True, exist_ok=True)

        d_type = getattr(self.diagram, 'diagram_type', 'class') or 'class'
        diagram_type_str = d_type.value if hasattr(d_type, 'value') else d_type

        data = {
            "version": "1.0",
            "name": self.diagram.name,
            "diagram_type": diagram_type_str,
            "nodes": [node.model_dump() for node in self.diagram.nodes],
            "connections": [conn.model_dump() for conn in self.diagram.connections],
            "uc_actors": [actor.model_dump() for actor in self.diagram.uc_actors],
            "uc_scenarios": [scenario.model_dump() for scenario in self.diagram.uc_scenarios],
            "uc_connections": [conn.model_dump() for conn in self.diagram.uc_connections],
        }
        with open(filepath, 'w', encoding='utf-8') as f:
            json.dump(data, f, indent=2, ensure_ascii=False)
        logger.info(f"Diagram saved successfully to {filepath}")
    return True
```

```

except PermissionError as e:
    logger.error(f"Permission denied when saving diagram to {filepath}: {e}")
    return False
except OSError as e:
    logger.error(f"OS error when saving diagram to {filepath}: {e}")
    return False
except Exception as e:
    logger.error(f"Unexpected error saving diagram to {filepath}: {e}", exc_info=True)
    return False

```

src/logiccraft/models/diagram_manager.py — метод load_from_file() : загружает диаграмму из JSON через json.load().

```

def load_from_file(self, filepath: str) -> bool:
    try:
        with open(filepath, 'r', encoding='utf-8') as f:
            data = json.load(f)

        nodes = [UMLNode(**node_data) for node_data in data.get("nodes", [])]
        connections = [UMLConnection(**conn_data) for conn_data in data.get("connections", [])]
        uc_actors = [UseCaseActor(**a) for a in data.get("uc_actors", [])]
        uc_scenarios = [UseCaseScenario(**s) for s in data.get("uc_scenarios", [])]
        uc_connections = [UseCaseConnection(**c) for c in data.get("uc_connections", [])]

        from .diagram import DiagramType
        diagram_type = DiagramType(data.get("diagram_type", "class"))

        self.diagram = UMLDiagram(
            name=data.get("name", "Untitled"),
            diagram_type=diagram_type,
            nodes=nodes,
            connections=connections,
            uc_actors=uc_actors,
            uc_scenarios=uc_scenarios,
            uc_connections=uc_connections,
        )
        return True
    except Exception as e:
        logger.error(f"Error loading diagram: {e}")
        return False

```

Для сериализации всей структуры проекта в строку формата JSON достаточно вызвать метод .model_dump_json() у корневого объекта:

Пример сохранения проекта в файл

```

def save_to_json(diagram: UMLDiagram, filepath: str) -> bool:
    try:
        data = SerializationService.serialize(diagram)
        with open(filepath, 'w', encoding='utf-8') as f:
            json.dump(data, f, indent=2, ensure_ascii=False)
        return True
    except Exception as e:
        logger.error(f"Error saving to JSON: {e}")
        return False

```

Процесс загрузки файла и восстановления полноценного графа объектов со всеми вложенными моделями (списками свойств, методов и связей) происходит в одну строчку через метод .model_validate_json():

```

# Пример загрузки проекта из файла
def load_from_json(filepath: str) -> Tuple[bool, UMLDiagram]:
    try:
        with open(filepath, 'r', encoding='utf-8') as f:
            data = json.load(f)
            diagram = SerializationService.deserialize(data)
        return True, diagram
    except Exception as e:
        logger.error(f"Error loading from JSON: {e}")
        return False, None

```

Таким образом, слой моделей данных в LogicCraft представляет собой изолированную, строго типизированную и легко сериализуемую библиотеку классов на чистом Python, готовую к работе в связке с любым интерфейсом или тестовым фреймворком.

Глава 5. Графический интерфейс

В этой главе мы перейдем от абстрактных моделей данных к визуальному слою приложения LogicCraft. Построение интерактивного графического редактора на PyQt6 — это задача, требующая глубокого понимания механики работы фреймворка, управления событиями мыши и оптимизации рендеринга.

Мы разберем, как с помощью графической подсистемы Qt Graphics View Framework превратить сухие JSON-структуры в отзывчивый, стильный и эффективный интерфейс с поддержкой Drag-and-Drop, динамических связей и адаптивных панелей управления.

5.1. QGraphicsScene — холст для диаграммы

Архитектура графического интерфейса LogicCraft опирается на связку трех классов Qt: **QGraphicsView** (окно просмотра), **QGraphicsScene** (интерактивный холст) и **QGraphicsItem** (визуальные элементы). Холст является центральным хабом, координирующим отображение сетки, размещение карточек классов и проведение связей.

Класс **DiagramScene** расширяет стандартную сцену, задавая фиксированные границы виртуального пространства (от -5000 до 5000 пикселей) и инициализируя базовый цвет фона из глобальных токенов **SceneStyle.BACKGROUND**:

```

class DiagramScene(QGraphicsScene):
    connection_ready = pyqtSignal(str, str, str, str)
    card_moved = pyqtSignal(str, float, float)
    actor_renamed = pyqtSignal(str, str) # actor_id, new_name
    scenario_renamed = pyqtSignal(str, str) # scenario_id, new_name

    def __init__(self, parent=None):
        super().__init__(parent)
        self.setBackgroundBrush(QBrush(QColor(SceneStyle.BACKGROUND)))
        self.setSceneRect(-5000, -5000, 10000, 10000)

        self.temp_line = None
        self.connection_source = None
        self.source_anchor = None
        self.connection_active = False

```

Сцена не просто пассивно хранит элементы, она берет на себя роль диспетчера сигналов верхнего уровня (**connection_ready**, **card_moved**), которые транслируются напрямую в главный контроллер приложения при завершении интерактивных действий пользователя.

5.2. UML-карточки: как нарисовать класс

Карточка класса (**UMLCard**) — это кастомный графический элемент, наследуемый от **QGraphicsRectItem**. В отличие от стандартных прямоугольников, **UMLCard** инкапсулирует сложную внутреннюю структуру: заголовок, секцию свойств, секцию методов и интерактивные порты (точки привязки).

Класс **UMLCard** находится в файле `src/logiccraft/view/widgets/uml_card.py`. Это графический элемент (**QGraphicsRectItem**), представляющий карточку класса UML на диаграмме.

```
class UMLCard(QGraphicsRectItem):
```

```
    ANCHOR_TOP = "top"
    ANCHOR_BOTTOM = "bottom"
    ANCHOR_LEFT = "left"
    ANCHOR_RIGHT = "right"

    HEADER_HEIGHT = 36
    RADIUS = 12
```

Отрисовка скруглений и заголовка через QPainterPath

Чтобы карточка выглядела современно, базовый метод `paint()` переопределяется. Вместо жестких углов используется отрисовка скругленного прямоугольника по контуру **QPainterPath** с радиусом скругления в 12 пикселей. Отрисовка разделена на два этапа: заливка шапки класса (**HEADER_HEIGHT = 36**) акцентным цветом и прорисовка тела карточки.

Механизм отрисовки:

1. `paint()` — кастомная отрисовка через **QPainter**:

- Тень : `drawRoundedRect()` с полупрозрачным фиолетовым цветом, смещённая на (2, 3).
 - Основной фон : `drawRoundedRect()` с белым фоном и скруглением **RADIUS = 12**.
 - Заголовок : **QPainterPath** — скругляются только верхние углы (левая и правая дуги `arcTo()`), нижние остаются прямыми.
- Цвет заголовка зависит от типа узла (`interface`, `enum`, `abstract_class` или стандартный).

```
def paint(self, painter: QPainter, option, widget=None):
```

```
    r = self.rect()
    radius = self.RADIUS
    h = self.HEADER_HEIGHT

    painter.setRenderHint(QPainter.RenderHint.Antialiasing)

    shadow_color = QColor(124, 58, 237, 25)
    painter.setPen(Qt.PenStyle.NoPen)
    painter.setBrush(QBrush(shadow_color))
    painter.drawRoundedRect(r.adjusted(2, 3, 2, 3), radius, radius)

    border_color = QColor(CardStyle.SELECTED_BORDER) if self._is_selected else QColor(CardStyle.BORDER)
    border_width = CardStyle.SELECTED_BORDER_WIDTH if self._is_selected else CardStyle.BORDER_WIDTH
    painter.setPen(QPen(border_color, border_width))
    painter.setBrush(QBrush(QColor(CardStyle.BACKGROUND)))
    painter.drawRoundedRect(r, radius, radius)

    node_type = getattr(self, 'node_type', 'class')
    if hasattr(node_type, 'value'):
        node_type = node_type.value

    if node_type == 'interface':
        header_color = QColor("#9B72F5") # Светло-фиолетовый для интерфейсов
    elif node_type == 'enum':
        header_color = QColor("#10B981") # Зелёный для enum
    elif node_type == 'abstract_class':
        header_color = QColor("#8B5CF6") # Средне-фиолетовый для абстрактных
    else:
        header_color = QColor(CardStyle.HEADER_BG) # Стандартный фиолетовый

    header_path = QPainterPath()
    header_path.moveTo(r.left() + radius, r.top())
    header_path.lineTo(r.right() - radius, r.top())
```

```

header_path.arcTo(r.right() - radius * 2, r.top(), radius * 2, radius * 2, 90, -90)
header_path.lineTo(r.right(), r.top() + h)
header_path.lineTo(r.left(), r.top() + h)
header_path.arcTo(r.left(), r.top(), radius * 2, radius * 2, 180, -90)
header_path.closeSubpath()

painter.setPen(Qt.PenStyle.NoPen)
painter.setBrush(QBrush(header_color))
painter.drawPath(header_path)

```

2. `_create_elements()` — создание визуальных элементов:

- `header_text (QGraphicsTextItem)` — имя класса.
- `attrs_text (QGraphicsTextItem)` — атрибуты.
- `methods_text (QGraphicsTextItem)` — методы.
- `divider2 (QGraphicsRectItem)` — разделитель между атрибутами и методами.

```

def _create_elements(self):
    self.header_text = QGraphicsTextItem(self)
    self.header_text.setDefaultTextColor(QColor(CardStyle.HEADER_TEXT))
    self.header_text.setFont(CardStyle.HEADER_FONT)

    self.attrs_text = QGraphicsTextItem("", self)
    self.attrs_text.setFont(CardStyle.ATTRS_FONT)
    self.attrs_text.setDefaultTextColor(QColor(CardStyle.ATTRS_TEXT))

    self.methods_text = QGraphicsTextItem("", self)
    self.methods_text.setFont(CardStyle.METHODS_FONT)
    self.methods_text.setDefaultTextColor(QColor(CardStyle.METHODS_TEXT))

    self.divider2 = QGraphicsRectItem(0, 0, self.rect().width(), 1, self)
    self.divider2.setBrush(QBrush(QColor(CardStyle.DIVIDER)))
    self.divider2.setPen(QPen(Qt.PenStyle.NoPen))

```

3. `update_content()` — расчёт высоты и позиционирование:

- Высота вычисляется по количеству атрибутов и методов.
- Заголовок центрируется по горизонтали.
- Атрибуты и методы размещаются с отступами.
- Разделитель показывается только если есть методы.

```

def update_content(self):
    n_attrs = len(self.attributes) if self.attributes else 1
    n_methods = len(self.methods) if self.methods else 1

    new_height = self.HEADER_HEIGHT + 8 + (n_attrs * 20) + 10 + (n_methods * 20) + 8
    new_height = max(110, new_height)
    width = self.rect().width()

    self.setRect(0, 0, width, new_height)

    node_type = getattr(self, 'node_type', 'class')
    if hasattr(node_type, 'value'):
        node_type = node_type.value

    if node_type == 'interface':
        stereotype = '«interface»'
        icon = '◇'
        name_style = 'font-style:italic; font-weight:bold;'
    elif node_type == 'enum':
        stereotype = '«enumeration»'
        icon = '≡'
        name_style = 'font-weight:bold;'
    elif node_type == 'abstract_class':
        stereotype = '«abstract»'

```

```

icon = '△'
name_style = 'font-style:italic; font-weight:bold;'
else:
    stereotype = ""
    icon = '◇'
    name_style = 'font-weight:bold;'

if stereotype:
    header_html = (
        f'<div style="text-align:center;">'
        f'<div style="font-size:9px; color:rgba(255,255,255,200); margin-bottom:2px;">'
        f'{stereotype}</div>'
        f'<div style="font-size:13px; {name_style}">'
        f'{icon} {self.name}</div>'
        f'</div>'
    )
else:
    header_html = (
        f'<div style="text-align:center; font-size:13px; {name_style}">'
        f'{icon} {self.name}</div>'
    )

self.header_text.setHtml(header_html)
tw = self.header_text.boundingRect().width()
th = self.header_text.boundingRect().height()
self.header_text.setPos((width - tw) / 2, (self.HEADER_HEIGHT - th) / 2)

attr_text = "\n".join(self.attributes) if self.attributes else ""
self.attrs_text.setPlainText(attr_text)
self.attrs_text.setPos(12, self.HEADER_HEIGHT + 8)

attr_h = self.attrs_text.boundingRect().height() if self.attributes else 0
divider_y = self.HEADER_HEIGHT + 8 + attr_h + 4

if self.methods:
    self.divider2.setRect(12, divider_y, width - 24, 1)
    self.divider2.show()
else:
    self.divider2.hide()

method_text = "\n".join(self.methods) if self.methods else ""
self.methods_text.setPlainText(method_text)
self.methods_text.setPos(12, divider_y + 6)

self._update_anchor_positions()
self.update()

```

5.3. Линии связей и стрелки

Связь между элементами (ConnectionLine) — один из самых уязвимых компонентов архитектуры с точки зрения утечек памяти и рассинхронизации геометрии. В LogicCraft связь наследуется от QGraphicsLineItem и хранит прямые ссылки на графические объекты source и target.

Линии связей и стрелки реализованы в двух файлах:

1. Линия связи — `_src/logiccraft/view/widgets/ connection_line.py`

Наследуется от QGraphicsLineItem .

Создаётся в `__init__()`:

```
class ConnectionSignals(QObject):
```

```
selected_changed = pyqtSignal(object, bool)
about_to_delete = pyqtSignal(object)
```

```
class ConnectionLine(QGraphicsLineItem):
```

```
    def __init__(self, source, target,
                 source_anchor: str = "right", target_anchor: str = "left",
                 connection_type: ConnectionType = ConnectionType.ASSOCIATION,
                 connection_id: str = None,
                 multiplicity: str = None,
                 name: str = None):
        super().__init__()

        self.id = connection_id or str(uuid.uuid4())
        self.source = source
        self.target = target
        self.source_anchor = source_anchor
        self.target_anchor = target_anchor
        self.connection_type = connection_type
        self.multiplicity = multiplicity or ""
        self.name = name or ""

        self.signals = ConnectionSignals()
```

Перо : QPen(QColor(ConnectionStyle.LINE_COLOR), ConnectionStyle.LINE_WIDTH).

Тип линии : для dependency, realization, uc_include, uc_extend — пунктир (DashLine).

```
pen = QPen(QColor(ConnectionStyle.LINE_COLOR), ConnectionStyle.LINE_WIDTH)
type_value = connection_type.value if hasattr(connection_type, 'value') else str(connection_type)
if type_value in ("dependency", "realization", "uc_include", "uc_extend"):
    pen.setStyle(Qt.PenStyle.DashLine)
```

```
self.setPen(pen)
```

Текстовые метки: multiplicity источника/цели, имя связи, стереотип для Include/Extend.

```
label_font = QFont("Menlo", 9)
self._source_mult_label = QGraphicsTextItem("", self)
self._source_mult_label.setFont(label_font)
self._source_mult_label.setDefaultTextColor(QColor("#7C3AED"))
```

```
self._target_mult_label = QGraphicsTextItem("", self)
self._target_mult_label.setFont(label_font)
self._target_mult_label.setDefaultTextColor(QColor("#7C3AED"))
```

```
self._name_label = QGraphicsTextItem("", self)
self._name_label.setFont(QFont("Inter", 9))
self._name_label.setDefaultTextColor(QColor("#6B7280"))
```

```
self._stereotype_label = QGraphicsTextItem("", self)
self._stereotype_label.setFont(QFont("Inter", 8, QFont.Weight.Medium))
self._stereotype_label.setDefaultTextColor(QColor("#7C3AED"))
self._update_stereotype_label()
```

Обновление позиции — метод update_position() пересчитывает линию при перемещении карточек.

```
def update_position(self):
    if not self.source or not self.target or not self.arrow_head:
        return
```

```

self.prepareGeometryChange()
p1 = self.source.get_anchor_point(self.source_anchor)
p2 = self.target.get_anchor_point(self.target_anchor)

line_vec = QLineF(p1, p2)
if line_vec.length() < 10:
    self.arrow_head.setVisible(False)
    self.setLine(line_vec)
    return

type_str = self.connection_type.value if hasattr(self.connection_type, 'value') else str(self.connection_type)
if "." in type_str:
    type_str = type_str.split(".")[1]
type_str = type_str.lower()

if type_str in ("uc_association", "association_none"):
    self.arrow_head.setVisible(False)
    self.setLine(line_vec)
else:
    self.arrow_head.setVisible(True)

angle = math.atan2(-line_vec.dy(), line_vec.dx())

offset = 6 if type_str in ("uc_include", "uc_extend", "dependency") else 12
p2_adj = QPointF(
    p2.x() - math.cos(angle) * offset,
    p2.y() + math.sin(angle) * offset
)

self.setLine(QLineF(p1, p2_adj))
self.arrow_head.setPos(p2)
self.arrow_head.set_direction(p2 - p1)

lp1 = self.mapFromScene(p1)
lp2 = self.mapFromScene(p2)
mid = QPointF((lp1.x() + lp2.x()) / 2, (lp1.y() + lp2.y()) / 2)

angle_labels = math.atan2(-line_vec.dy(), line_vec.dx())
cos_a = math.cos(angle_labels)
sin_a = math.sin(angle_labels)

self._source_mult_label.setPos(lp1.x() + cos_a * 15 + 5, lp1.y() - sin_a * 15 - 15)
self._target_mult_label.setPos(lp2.x() - cos_a * 35, lp2.y() + sin_a * 15 - 15)

self._name_label.setPos(mid.x() - self._name_label.boundingRect().width() / 2, mid.y() - 18)

sl_w = self._stereotype_label.boundingRect().width()
lbl_offset = -32 if self._name_label.isVisible() else -18
self._stereotype_label.setPos(mid.x() - sl_w / 2, mid.y() + lbl_offset)

```

Сигналы : подключается к position_changed source и target карточек.

```

if hasattr(source, 'signals'):
    source.signals.position_changed.connect(self.update_position)
if hasattr(target, 'signals'):
    target.signals.position_changed.connect(self.update_position)

```

2. Наконечник стрелки —src/logiccraft/view/widgets/arrow_head.py

Наследуется от QGraphicsPathItem).

class ArrowHead(QGraphicsPolygonItem):

Форма строится через QPainterPath в _update_shape() :

def _update_shape(self):

size = ArrowStyle.SIZE

type_str = self._get_clean_type()

normal_pen = QPen(QColor(ArrowStyle.COLOR), ArrowStyle.WIDTH_NORMAL)

thin_pen = QPen(QColor(ArrowStyle.COLOR), ArrowStyle.WIDTH_THIN)

background_brush = QBrush(QColor("#FFFFFF"))

path = QPainterPath()

Наследование/реализация : треугольник с заливкой (пустой треугольник: background_brush — белый).

if type_str in ("inheritance", "realization"):

path.moveTo(size, 0)

path.lineTo(0, -size * 0.6)

path.lineTo(0, size * 0.6)

path.closeSubpath()

self.setPath(path)

self.setBrush(background_brush)

self.setPen(normal_pen)

Композиция: закрашенный ромб/треугольник.

elif type_str == "composition":

path.moveTo(size, 0)

path.lineTo(size / 2, -size * 0.6)

path.lineTo(0, 0)

path.lineTo(size / 2, size * 0.6)

path.closeSubpath()

self.setPath(path)

self.setBrush(QBrush(QColor(ArrowStyle.COLOR)))

self.setPen(thin_pen)

Агрегация: пустой ромб/треугольник.

elif type_str == "aggregation":

path.moveTo(size, 0)

path.lineTo(size / 2, -size * 0.6)

path.lineTo(0, 0)

path.lineTo(size / 2, size * 0.6)

path.closeSubpath()

self.setPath(path)

self.setBrush(background_brush)

self.setPen(normal_pen)

Зависимость/Include/Extend: открытый уголок без заливки.

elif type_str in ("dependency", "uc_include", "uc_extend", "include", "extend"):

path.moveTo(0, -size * 0.5)

path.lineTo(size, 0)

path.lineTo(0, size * 0.5)

self.setPath(path)

self.setBrush(QBrush(Qt.BrushStyle.NoBrush))

self.setPen(normal_pen)

Ассоциация/Interaction (строки 105–112): закрашенный треугольник.

```
else: # association, interaction
    path.moveTo(size, 0)
    path.lineTo(0, -size * 0.5)
    path.lineTo(0, size * 0.5)
    path.closeSubpath()
    self.setPath(path)
    self.setBrush(QBrush(QColor(ArrowStyle.COLOR)))
    self.setPen(thin_pen)
```

Вращение — `update_rotation()`: вычисляет угол через `atan2` и применяет `setRotation()`.

```
def update_rotation(self):
    if hasattr(self, 'direction') and self.direction:
        angle = math.degrees(math.atan2(self.direction.y(), self.direction.x()))
        self.setRotation(angle)
```

5.4. Точки привязки и создание связей мышью

Для удобства пользователя создание связей реализовано интерактивно: при наведении мыши на карточку на её гранях (Top, Bottom, Left, Right) проявляются маленькие окружности — порты (`AnchorPoint`).

Механизм интерактивного связывания устроен следующим образом:

1. **Захват порта:** Когда пользователь зажимает левую кнопку мыши над `AnchorPoint`, порт переводит `DiagramScene` в режим построения связи, передавая ей идентификатор карточки-источника.
2. **Трассировка временной линии:** В процессе перемещения курсора по экрану (`mouseMoveEvent`), сцена создает объект `QGraphicsLineItem` с типом линии `Qt.PenStyle.DashLine` (пунктир). Конец этой линии непрерывно следует за координатами мыши.
3. **Фиксация на цели:** Во время протаскивания пунктирной линии сцена временно подсвечивает (`_show_all_anchors`) порты всех остальных карточек на холсте. Если отпускание мыши (`mouseReleaseEvent`) происходит над портом другой карточки, сцена генерирует сигнал `connection_ready`, уничтожает временную пунктирную линию и передает управление контроллеру для создания реального объекта связи.

Точки привязки — `src/logiccraft/view/widgets/anchora_point.py`

- Создаются в `UMLCard._create_anchors()` (4 точки: top, bottom, left, right). (`src/logiccraft/view/widgets/uml_card.py`),

```
def create_elements(self):
    self.header_text = QGraphicsTextItem(self)
    self.header_text.setDefaultTextColor(QColor(CardStyle.HEADER_TEXT))
    self.header_text.setFont(CardStyle.HEADER_FONT)
```

- `mousePressEvent()`: при нажатии ЛКМ вызывает `scene.start_connection()`.

```
def mousePressEvent(self, event):
    if event.button() == Qt.MouseButton.LeftButton:
        self._drag_start = self.scenePos()
        self.setCursor(Qt.CursorShape.ClosedHandCursor)
        scene = self.scene()
        if scene and hasattr(scene, 'start_connection'):
            scene.start_connection(self.parent_card, self.anchor_name)
        event.accept()
```

- `mouseMoveEvent()`: обновляет временную линию через `scene.update_temp_line()`.

```
def mouseMoveEvent(self, event):
    if event.buttons() == Qt.MouseButton.LeftButton and self._drag_start:
        current_pos = self.mapToScene(event.pos())
        scene = self.scene()
        if scene and hasattr(scene, 'update_temp_line'):
            scene.update_temp_line(current_pos)
```


`super().mousePressEvent(event)`

- `mouseReleaseEvent()`: ищет целевую точку привязки и вызывает `scene.finish_connection()` или `scene.cancel_connection()`.

`def mouseReleaseEvent(self, event):`

```
if event.button() == Qt.MouseButton.LeftButton and self._drag_start:
    self.setCursor(Qt.CursorShape.ArrowCursor)
    end_pos = self.mapToScene(event.pos())

    search_rect = self._create_search_rect(end_pos)
    items = self.scene().items(search_rect)

    target_anchor = None
    for item in items:
        if isinstance(item, AnchorPoint) and item.parent_card != self.parent_card:
            target_anchor = item
            break

    scene = self.scene()
    if target_anchor and hasattr(scene, 'finish_connection'):
        scene.finish_connection(target_anchor.parent_card, target_anchor.anchor_name)
    elif hasattr(scene, 'cancel_connection'):
        scene.cancel_connection()

    self._drag_start = None
    super().mouseReleaseEvent(event)
```

2. Сцена — `_src/logiccraft/view/scenes/diagram_scene.py`

- `start_connection()`: создаёт синюю пунктирную линию (`temp_line`) от точки привязки источника.

`def start_connection(self, source_card, anchor_name):`

```
"""Появление синего пунктира при начале тяги"""
self.connection_source = source_card
self.source_anchor = anchor_name
self.connection_active = True

start_pos = source_card.get_anchor_point(anchor_name)
self.temp_line = QGraphicsLineItem(QLineF(start_pos, start_pos))
self.temp_line.setPen(QPen(QColor(SceneStyle.TEMP_LINE_COLOR), 2, Qt.PenStyle.DashLine))

self.temp_line.setAcceptedMouseButtons(Qt.MouseButton.NoButton)
self.temp_line.setZValue(1000)
self.addItem(self.temp_line)

self._show_all_anchors(except_card=source_card)
```

- `mouseMoveEvent()`: тянет линию за мышью, обновляя P2.

`def mouseMoveEvent(self, event):`

```
if self.connection_active and self.temp_line:
    line = self.temp_line.line()
    line.setP2(event.scenePos())
    self.temp_line.setLine(line)
    super().mouseMoveEvent(event)
```

- `finish_connection()`: удаляет пунктир, emits сигнал `connection_ready` с ID source/target и якорями.

`def finish_connection(self, target_card, target_anchor):`

```
if not self.connection_active:
    return

if self.temp_line:
    self.removeItem(self.temp_line)
    self.temp_line = None

if self.connection_source and target_card and self.connection_source != target_card:
    self.connection_ready.emit()
```



```

        self.connection_source.id,
        target_card.id,
        self.source_anchor,
        target_anchor
    )

    self.connection_source = None
    self.source_anchor = None
    self.connection_active = False

    self._hide_all_anchors()

```

- `cancel_connection()`: отмена тяги.

```

def cancel_connection(self):
    """Отмена тяги"""
    if self.temp_line:
        self.removeItem(self.temp_line)
        self.temp_line = None
    self.connection_source = None
    self.source_anchor = None
    self.connection_active = False

    self._hide_all_anchors()

```

3. Обработка сигнала — `MainWindow._on_connection_ready()` (`src/logiccraft/view/main_window.py`,)
 Получает сигнал из сцены и вызывает `controller.add_connection()` или `controller.add_uc_connection()`.

```

def _on_connection_ready(self, source_id, target_id, source_anchor, target_anchor):
    conn_type = getattr(self, '_default_connection_type', 'association')
    uc_types = {"uc_association", "uc_include", "uc_extend"}
    all_uc_ids = set(self.uc_actor_map) | set(self.uc_scenario_map)
    if source_id in all_uc_ids or target_id in all_uc_ids or conn_type in uc_types:
        uc_type = conn_type if conn_type in uc_types else "uc_association"
        self.controller.add_uc_connection(source_id, target_id, uc_type,
                                          source_anchor, target_anchor)
    else:
        self.controller.add_connection(source_id, target_id, conn_type, source_anchor, target_anchor)

```

5.5. Панель инструментов (Toolbox) и панель свойств (Properties)

Графический холст работает в тесной связке со вспомогательными боковыми панелями, унаследованными от базового `QWidget`.

Левая панель инструментов (Toolbox (панель инструментов): `src/logiccraft/view/panels/toolbox_panel.py` — класс `ToolboxPanel`)

ниже представлена часть кода, полную версию можно посмотреть на GitHub (<https://github.com/Trewasasha/LogicCraft>)
`class` `ToolboxPanel(QWidget)`:

```

add_element_requested = pyqtSignal(str) # node_type или uc_actor / uc_scenario
set_connection_mode = pyqtSignal(str)  # connection_type

def __init__(self, parent=None):
    super().__init__(parent)
    self.setObjectName("ToolboxPanel")
    self.setFixedWidth(200)
    self._setup_ui()

def _setup_ui(self):
    outer = QVBoxLayout(self)
    outer.setContentsMargins(0, 0, 0, 0)
    outer.setSpacing(0)

    scroll = QScrollArea()
    scroll.setWidgetResizable(True)

```

```

scroll.setFrameShape(QFrame.Shape.NoFrame)
scroll.setHorizontalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBarAlwaysOff)
outer.addWidget(scroll)

container = QWidget()
scroll.setWidget(container)

layout = QVBoxLayout(container)
layout.setContentsMargins(12, 12, 12, 12)
layout.setSpacing(16)
layout.setAlignment(Qt.AlignmentFlag.AlignTop)

```

Представляет собой меню выбора текущего режима работы. Кнопки панели (**ToolboxButton**) кастомизированы под общий плоский стиль интерфейса. Тулбокс имеет встроенный фильтр контекста: метод **set_diagram_type(diagram_type)** динамически скрывает или показывает группы инструментов. Если пользователь переключается на Use Case диаграмму, секции "Классы" и "Связи классов" скрываются, уступая место "Актерам" и "Сценариям".

Правая панель свойств (Properties (панель свойств): src/logiccraft/view/panels/properties_panel.py — класс PropertiesPanel)

```

class PropertiesPanel(QWidget):
    name_changed = pyqtSignal(str, str)      # card_id, new_name
    type_changed = pyqtSignal(str, str)      # card_id, new_type
    attributes_changed = pyqtSignal(str, list) # card_id, attributes
    methods_changed = pyqtSignal(str, list)   # card_id, methods
    delete_requested = pyqtSignal(str)       # card_id
    # Use Case сигналы
    uc_actor_name_changed = pyqtSignal(str, str) # actor_id, new_name
    uc_scenario_name_changed = pyqtSignal(str, str) # scenario_id, new_name
    uc_scenario_desc_changed = pyqtSignal(str, str) # scenario_id, new_description
    uc_element_delete_requested = pyqtSignal(str, str) # element_id, element_type

```

Отвечает за детальное редактирование выделенного элемента диаграммы. Она содержит поля ввода (**QLineEdit**), списки (**QListWidget**) и выпадающие меню (**QComboBox**) для управления типами узла, свойствами и методами.

Защита от дребезга и бесконечных циклов через QTimer(src/logiccraft/view/panels/properties_panel.py в классе PropertiesPanel:)

1. Имя класса : _name_timer — setSingleShot(True), задержка 500 мс, подключен к _emit_name_changed.

```

if not hasattr(self, '_name_timer'):
    self._name_timer = QTimer()
    self._name_timer.setSingleShot(True)
    self._name_timer.timeout.connect(self._emit_name_changed)
self._name_timer.start(500)

```

2. Имя Use Case : _uc_name_timer — аналогично, 500 мс, подключен к _emit_uc_name_changed.

```

if not hasattr(self, '_uc_name_timer'):
    self._uc_name_timer = QTimer()
    self._uc_name_timer.setSingleShot(True)
    self._uc_name_timer.timeout.connect(self._emit_uc_name_changed)
self._uc_name_timer.start(500)

```

3. Описание Use Case : _uc_desc_timer — аналогично, 500 мс, подключен к _emit_uc_desc_changed.

```

if not hasattr(self, '_uc_desc_timer'):
    self._uc_desc_timer = QTimer()
    self._uc_desc_timer.setSingleShot(True)
    self._uc_desc_timer.timeout.connect(self._emit_uc_desc_changed)
self._uc_desc_timer.start(500)

```

Так как ввод текста пользователем происходит посимвольно, прямая отправка сигналов об изменении имени на каждый клик клавиатуры перегружала бы слой бизнес-логики и сервис истории (Undo/Redo). Для решения этой проблемы в **PropertiesPanel** интегрирован паттерн Debounce (задержка) на базе **QTimer**:

5.6. Система стилей: theme.py и style.qss

Визуальный облик LogicCraft формируется единой дизайн-системой, разделенной на два уровня: программные константы и каскадные таблицы стилей Qt (QSS).

1. **theme.py:** Содержит строго именованные константы цветов и размеров, используемые непосредственно при отрисовке в методах `paint()`. Например, `SceneStyle.BACKGROUND = "#F0EFFE"`, фиолетовые акценты для выделения (`#7C3AED`) и нейтральные оттенки рамок. Это гарантирует, что геометрические элементы сцены и карточек всегда синхронизированы по цветовой гамме.

ниже представлены не все стили полную версию можно посмотреть на GitHub(<https://github.com/Trewasasha/LogicCraft>)

```
@dataclass(frozen=True)
class _ConnectionStyle:
    LINE_COLOR: str = "#9B72F5"
    LINE_WIDTH: int = 2
    SELECTED_COLOR: str = "#7C3AED"
    SELECTED_WIDTH: int = 3

@dataclass(frozen=True)
class _SceneStyle:
    BACKGROUND: str = "#F0EFFE"
    GRID_COLOR: str = "#E0D9FC"
    GRID_WIDTH: float = 0.5
    GRID_STEP: int = 50
    TEMP_LINE_COLOR: str = "#7C3AED"

@dataclass(frozen=True)
class _AnchorStyle:
    NORMAL_COLOR: str = "#9B72F5"
    HOVER_COLOR: str = "#7C3AED"
    BORDER_COLOR: str = "#FFFFFF"
    BORDER_WIDTH: float = 1.5
    HOVER_SCALE: float = 1.2
```

2. **style.qss:** Файл каскадных стилей, применяемый ко всему приложению на старте. Он стилизует стандартные виджеты Qt (кнопки, панели, доки), создавая единый монолитный дизайн:

ниже представлена часть кода, полную версию можно посмотреть на GitHub(<https://github.com/Trewasasha/LogicCraft>)

```
QMainWindow {
    background-color: #F0EFFE;
}

QWidget {
    background-color: #F0EFFE;
    color: #1F1F1F;
    font-family: "Inter", "Segoe UI", sans-serif;
    font-size: 13px;
}

/* === Toolbar === */
QToolBar {
    background-color: #FFFFFF;
    border-bottom: 1px solid #E5E0F8;
    spacing: 6px;
    padding: 4px 8px;
}
```

Использование QSS позволяет полностью отделить логику компоновки интерфейса от его декоративного оформления.

5.7. Масштабирование и панорамирование

Комфортная работа с большими архитектурными диаграммами невозможна без удобной навигации. Эти функции реализуются на уровне окна просмотра `QGraphicsView`.

- **Панорамирование (Hand Scroll):** При зажатии средней кнопки мыши (или пробела) курсор переводится в режим руки (`Qt.CursorShape.OpenHandCursor`), и движение мыши преобразуется в смещение ползунков прокрутки холста через вызов метода `horizontalScrollBar().setValue()`.

- **Масштабирование (Zoom):** Реализуется переопределением метода `wheelEvent`. Изменение масштаба привязано к позиции курсора мыши, чтобы пользователь мог приближать конкретный интересующий его узел:

Масштабирование и панорамирование реализованы в классе `DiagramView` в файле `src/logiccraft/view/main_window.py` ниже представлен не весь код полную версию можно посмотреть на GitHub(<https://github.com/Trewasasha/LogicCraft>)

Методы класса `DiagramView`:

- `__init__` — инициализация `scale_factor=1.15`, `_zoom_level=100`, `_min_zoom=20`, `_max_zoom=400`, флаги панорамирования

```
def __init__(self, scene: DiagramScene, parent=None):
    super().__init__(scene, parent)
    self.setRenderHint(QPainter.RenderHint.Antialiasing)
    self.setDragMode(QGraphicsView.DragMode.RubberBandDrag)
    self.setTransformationAnchor(QGraphicsView.ViewportAnchor.AnchorUnderMouse)
    self.setResizeAnchor(QGraphicsView.ViewportAnchor.AnchorUnderMouse)
    self.scale_factor = 1.15
    self._zoom_level = 100 # текущий масштаб в %
    self._min_zoom = 20
    self._max_zoom = 400
    self._panning = False
    self._pan_start = None
    self.main_window = parent
```

- `wheelEvent` — Ctrl+колесо зуммирует, без Ctrl прокручивает

```
def wheelEvent(self, event):
    """Zoom колесом мыши + Ctrl, прокрутка без Ctrl"""
    if event.modifiers() == Qt.KeyboardModifier.ControlModifier:
        delta = event.angleDelta().y()
        if delta > 0:
            self._zoom_in()
        else:
            self._zoom_out()
    else:
        super().wheelEvent(event)
```

- `mousePressEvent` — средняя кнопка начинает панорамирование

```
def mousePressEvent(self, event):
    """Средняя кнопка — панорамирование"""
    if event.button() == Qt.MouseButton.MiddleButton:
        self._panning = True
        self._pan_start = event.position().toPoint()
        self.setCursor(Qt.CursorShape.ClosedHandCursor)
        event.accept()
        return
    super().mousePressEvent(event)
```

- `mouseReleaseEvent` — отпускание средней кнопки останавливает панорамирование

```
def mouseReleaseEvent(self, event):
    if event.button() == Qt.MouseButton.MiddleButton:
        self._panning = False
        self.setCursor(Qt.CursorShape.ArrowCursor)
        event.accept()
```

```

    return
    super().mouseReleaseEvent(event)
    • reset\_zoom — сброс до 100%
def reset_zoom(self):
    """Сбросить масштаб до 100%"""
    self.set_zoom(100)
    • fit\_in\_view\_all — вписать диаграмму в экран
В MainWindow:
    • горячие клавиши Ctrl+=, Ctrl+-, Ctrl+0
if event.modifiers() == Qt.KeyboardModifier.ControlModifier and event.key() == Qt.Key.Key_Equal:
    self.view._zoom_in()
    return True
if event.modifiers() == Qt.KeyboardModifier.ControlModifier and event.key() == Qt.Key.Key_Minus:
    self.view._zoom_out()
    return True
if event.modifiers() == Qt.KeyboardModifier.ControlModifier and event.key() == Qt.Key.Key_0:
    self.view.reset_zoom()
    return True

```

5.8. Сетка и привязка к сетке

Диаграмма выглядит профессионально только в том случае, если элементы выровнены относительно друг друга.

[DiagramScene](#) берет на себя автоматическую отрисовку координатной сетки через метод [drawBackground](#).

Сетка рисуется динамически в пределах видимого пользователю прямоугольника ([rect](#)), что исключает трату ресурсов на рендеринг бесконечного холста

ниже представлен не весь код полную версию можно посмотреть на GitHub(<https://github.com/Trewasasha/LogicCraft>)

[src/logiccraft/view/scenes/diagram_scene.py](#)

[DiagramScene.drawBackground\(\)](#) — отрисовка сетки

```

def drawBackground(self, painter, rect):
    super().drawBackground(painter, rect)
    pen = QPen(QColor(SceneStyle.GRID_COLOR), SceneStyle.GRID_WIDTH)
    painter.setPen(pen)

    left = int(rect.left()) - (int(rect.left()) % SceneStyle.GRID_STEP)
    top = int(rect.top()) - (int(rect.top()) % SceneStyle.GRID_STEP)

    for x in range(left, int(rect.right()), SceneStyle.GRID_STEP):
        painter.drawLine(x, int(rect.top()), x, int(rect.bottom()))
    for y in range(top, int(rect.bottom()), SceneStyle.GRID_STEP):
        painter.drawLine(int(rect.left()), y, int(rect.right()), y)

```

[src/logiccraft/view/theme.py](#)

[_SceneStyle](#) — константы сетки

```

@dataclass(frozen=True)
class _ConnectionStyle:
    LINE_COLOR: str = "#9B72F5"
    LINE_WIDTH: int = 2
    SELECTED_COLOR: str = "#7C3AED"
    SELECTED_WIDTH: int = 3

```

```

@dataclass(frozen=True)
class _SceneStyle:
    BACKGROUND: str = "#F0EFFE"
    GRID_COLOR: str = "#E0D9FC"
    GRID_WIDTH: float = 0.5

```

```
GRID_STEP: int = 50
```

```
TEMP_LINE_COLOR: str = "#7C3AED"
```

```
@dataclass(frozen=True)
```

```
class _AnchorStyle:
```

```
    NORMAL_COLOR: str = "#9B72F5"
```

```
    HOVER_COLOR: str = "#7C3AED"
```

```
    BORDER_COLOR: str = "#FFFFFF"
```

```
    BORDER_WIDTH: float = 1.5
```

```
    HOVER_SCALE: float = 1.2
```

src/logiccraft/config/settings.py

[AppSettings](#) — настройки grid_size, show_grid (строки 20–21, 66–71)

```
def _load_settings(self) -> Dict[str, Any]:
```

```
    """Загрузить настройки из файла"""
```

```
    default_settings = {
```

```
        "theme": "light",
```

```
        "language": "python",
```

```
        "grid_size": 50,
```

```
        "show_grid": True,
```

```
        "auto_save": True,
```

```
        "auto_save_interval": 300, # секунд
```

```
        "last_open_path": str(Path.home()),
```

```
        "window_width": 1200,
```

```
        "window_height": 800,
```

```
        "font_size": 10
```

```
}
```

```
@property
```

```
def show_grid(self) -> bool:
```

```
    return self.settings.get("show_grid", True)
```

```
@property
```

```
def grid_size(self) -> int:
```

```
    return self.settings.get("grid_size", 50)
```

При перемещении карточки (itemChange), ее новые координаты округляются до ближайшего шага сетки. Таким образом, карточка мягко "магнитится" к линиям при перетаскивании.

5.9. Выделение нескольких объектов

LogicCraft поддерживает как одиночное, так и групповое выделение элементов. Для включения группового выделения в [QGraphicsView](#) активируется режим резиновой ленты ([QGraphicsView.RubberBandSelectionMode.IntersectsItemShape](#)).

Когда пользователь зажимает левую кнопку мыши на пустом пространстве холста и растягивает прямоугольник рамки, Qt автоматически определяет все элементы, попавшие в зону выделения. Выделенные карточки визуально меняют цвет своей рамки на акцентный фиолетовый. При последующем Drag-and-Drop действии все выделенные узлы перемещаются синхронно, а связанные с ними линии автоматически пересчитывают свои геометрические векторы прямо в процессе движения.

- `DiagramView.__init__` — `setDragMode(QGraphicsView.DragMode.RubberBandDrag)` — режим выделения рамкой(src/logiccraft/view/main_window.py)

```
self.setDragMode(QGraphicsView.DragMode.RubberBandDrag)
```

- `DiagramScene.on_card_moved()` — групповое перемещение выделенных `UMLCard`, `ActorWidget`, `ScenarioWidget`(src/logiccraft/view/scenes/diagram_scene.py)

```
def on_card_moved(self, card):
```

```
    from ..widgets.uml_card import UMLCard
```

```
    from ..widgets.actor_widget import ActorWidget
```

```
    from ..widgets.scenario_widget import ScenarioWidget
```

```
grid = SceneStyle.GRID_STEP
```

```
selected_items = [  
    item for item in self.selectedItems()  
    if isinstance(item, (UMLCard, ActorWidget, ScenarioWidget))  
]  
targets = selected_items if len(selected_items) > 1 else [card]
```

```
for c in targets:  
    x = round(c.pos().x() / grid) * grid  
    y = round(c.pos().y() / grid) * grid  
    if c.pos().x() != x or c.pos().y() != y:  
        c.setPos(x, y)  
    if isinstance(c, UMLCard):  
        self.card_moved.emit(c.id, c.pos().x(), c.pos().y())  
• MainWindow._on_delete_selected() — удаление всех выделенных элементов(src/logiccraft/view/widgets/uml_card.py)
```

```
def _align_center_v(self):  
    cards = self._get_selected_cards()  
    if len(cards) < 2: return  
    avg_y = sum(c.pos().y() + c.rect().height() / 2 for c in cards) / len(cards)  
    for c in cards:  
        ny = avg_y - c.rect().height() / 2  
        c.setPos(c.pos().x(), ny)  
        self.controller.update_card(c.id, x=c.pos().x(), y=ny)
```

```
• MainWindow._on_duplicate_selected() — дублирование выделенных  
карточек(src/logiccraft/view/widgets/actor_widget.py)
```

```
def _on_duplicate(self):  
    selected = [item for item in self.scene.selectedItems() if isinstance(item, UMLCard)]  
    for card in selected:  
        self.controller.duplicate_card(card.id)
```

```
• MainWindow._on_select_all() — выделение всех элементов(src/logiccraft/view/widgets/scenario_widget.py)
```

```
def _on_select_all(self):  
    for item in self.scene.items():  
        item.setSelected(True)
```

```
• MainWindow._on_connection_selected() — выделение связей  
ConnectionLine(src/logiccraft/view/widgets/connection_line.py)
```

```
def _on_toolbox_set_connection_type(self, connection_type: str):  
    selected_connections = [  
        item for item in self.scene.selectedItems()  
        if isinstance(item, ConnectionLine)  
    ]  
    if selected_connections:  
        for conn in selected_connections:  
            self.controller.update_connection_type(conn.id, connection_type)  
            conn.set_connection_type(connection_type)  
            conn.update_position()  
            self.update_status(f"Тип связи изменён: {connection_type}")  
    else:  
        self._default_connection_type = connection_type  
        self.update_status(f"Следующая связь: {connection_type}")
```


В архитектурном паттерне MVC компонент **Controller** выполняет роль «регулирующего», который изолирует графический интерфейс (View) от внутренней кухни приложения (Model и Services). В LogicCraft за это отвечает класс **DiagramController**. Он перехватывает сигналы от графических карточек и холста, транслирует их в вызовы бизнес-логики через **DiagramManager**, управляет снимками состояний с помощью **HistoryService** и возвращает результат обратно в UI через собственные сигналы.

В этой главе мы детально разберем реализацию центрального управляющего узла приложения, механизмы атомарного изменения диаграммы, а также построение полноценной системы отмены и повтора действий (Undo/Redo) на базе снимков состояний глубоких моделей Pydantic.

6.1. DiagramController — главный контроллер

Класс **DiagramController** наследуется от **QObject**, что позволяет ему использовать нативный механизм сигналов и слотов PyQt6 для асинхронного и потокобезопасного взаимодействия с графической оболочкой.

Контроллер инкапсулирует внутри себя три важнейших компонента:

1. **DiagramManager** — для CRUD-операций над структурами данных диаграммы.
2. **HistoryService** — для фиксации и отката изменений.
3. **DiagramEngine** — для выполнения графовых алгоритмов и валидации.

ниже представлена часть кода, полную версию можно посмотреть на GitHub(<https://github.com/Trewasasha/LogicCraft>):
src/logiccraft/controllers/diagram_controller.py

```
class DiagramController(QObject):
    card_added = pyqtSignal(object)
    card_removed = pyqtSignal(str)
    card_updated = pyqtSignal(object)

    connection_added = pyqtSignal(object)
    connection_removed = pyqtSignal(str)
    connection_updated = pyqtSignal(str)

    diagram_cleared = pyqtSignal()
    diagram_loaded = pyqtSignal()

    status_changed = pyqtSignal(str)
    error_occurred = pyqtSignal(str)
    undo_redo_changed = pyqtSignal()

    uc_actor_added = pyqtSignal(object) # UseCaseActor
    uc_actor_removed = pyqtSignal(str)
    uc_scenario_added = pyqtSignal(object) # UseCaseScenario
    uc_scenario_removed = pyqtSignal(str)
    uc_connection_added = pyqtSignal(object) # UseCaseConnection
    uc_connection_removed = pyqtSignal(str)
```

Сигналы спроектированы таким образом, чтобы передавать измененный объект целиком (например, `card_added = pyqtSignal(object)` передает инстанс `UMLNode`), либо транслировать строковый уникальный идентификатор `UUID(card_removed = pyqtSignal(str))`. Это позволяет View мгновенно находить нужный графический виджет на сцене и удалять или перерисовывать его.

6.2. Добавление, редактирование и удаление элементов

Любая манипуляция с визуальными сущностями на холсте (клик мышкой, редактирование имени класса внутри текстового поля карточки, удаление через кнопку Delete) обязательно проходит сквозь методы-мутаторы контроллера. Рассмотрим сокращенный исходный код методов, отвечающих за жизненный цикл UML-узлов и связей(полный код тут:<https://github.com/Trewasasha/LogicCraft>):


```

def add_connection(self, source_id: str, target_id: str, connection_type: str,
                   source_anchor: str = "right", target_anchor: str = "left") -> Optional[UMLConnection]:
    try:
        conn = self.manager.add_connection(source_id, target_id, connection_type, source_anchor, target_anchor)
        if conn:
            self.connection_added.emit(conn)
            self._save_state()
            return conn
        return None
    except Exception as e:
        self.error_occurred.emit(f"Ошибка создания связи: {e}")
        return None

def update_connection_type(self, connection_id: str, new_type: str) -> bool:
    try:
        if self.manager.update_connection_type(connection_id, new_type):
            self.connection_updated.emit(connection_id)
            self._save_state()
            return True
        return False
    except Exception as e:
        self.error_occurred.emit(f"Ошибка обновления типа: {e}")
        return False

def update_connection_properties(self, connection_id: str,
                                new_type: str = None,
                                multiplicity: str = None,
                                name: str = None) -> bool:
    try:
        conn = self.manager.get_connection_by_id(connection_id)
        if not conn:
            return False
        if new_type is not None:
            self.manager.update_connection_type(connection_id, new_type)
        if multiplicity is not None:
            conn.multiplicity = multiplicity
        if name is not None:
            conn.name = name
        self.connection_updated.emit(connection_id)
        self._save_state()
        return True
    except Exception as e:
        self.error_occurred.emit(f"Ошибка обновления связи: {e}")
        return False

def get_connection_model(self, connection_id: str) -> Optional[UMLConnection]:
    for conn in self.manager.diagram.connections:
        if conn.id == connection_id:
            return conn
    return None

def remove_connection(self, connection_id: str) -> bool:
    if self.manager.remove_connection(connection_id):
        self.connection_removed.emit(connection_id)
        self._save_state()

```

```

        return True
    return False

    def update_uc_scenario(self, scenario_id: str, name: str = None,
                           x: float = None, y: float = None) -> bool:
        """Обновить сценарий. Позицию обновляет без сохранения в историю — используй on_uc_scenario_move_finished"""
        for scenario in self.manager.diagram.uc_scenarios:
            if scenario.id == scenario_id:
                if name is not None:
                    scenario.name = name
                    self._save_state() # сохраняем только при изменении имени
                if x is not None:
                    scenario.x = x
                if y is not None:
                    scenario.y = y
                return True
        return False

def on_uc_scenario_move_finished(self, scenario_id: str, x: float, y: float):
    """Вызывается по завершению перетаскивания сценария (mouseRelease)"""
    for scenario in self.manager.diagram.uc_scenarios:
        if scenario.id == scenario_id:
            scenario.x = x
            scenario.y = y
            self._save_state()
            return True
    return False

```

6.3. Паттерн Command и реализация Undo/Redo

Классическая реализация паттерна Command (Команда) требует создания отдельного класса под каждое действие пользователя: `AddNodeCommand`, `MoveNodeCommand`, `DeleteConnectionCommand`. Каждый такой класс обязан хранить в себе точечный снимок дельты изменений и реализовывать методы `execute()` и `unexecute()`. В сложных визуальных редакторах с разветвленными графовыми моделями этот подход порождает колоссальное количество шаблонного кода (Boilerplate) и усложняет поддержку.

В LogicCraft применен альтернативный, современный подход — **хранение полной иммутабельной истории состояний системы (State Snapshot-based Undo/Redo)**, оптимизированной за счет инкрементальной валидации и сериализации метаданных Pydantic. Так как базовая модель `UMLDiagram` является валидируемым Pydantic-объектом, мы можем мгновенно делать глубокие копии текущего состояния графа через стандартный вызов `.model_copy(deep=True)`. Управление стеком этих состояний инкапсулировано в `HistoryService`:

```
logger = logging.getLogger(__name__)
```

```

class HistoryService(QObject):
    history_changed = pyqtSignal(bool, bool) # (can_undo, can_redo)
    state_restored = pyqtSignal(object)      # Передает UMLDiagram во View
    validation_failed = pyqtSignal(str)      # Сообщение об ошибке в структуре

    def __init__(self, max_history: int = 50, initial_state: UMLDiagram = None):
        super().__init__()
        self._stack: List[UMLDiagram] = []
        self._current_index: int = -1
        self._max_history = max_history
        self._lock = Lock()
        self._is_restoring = False # Флаг, блокирующий запись во время Undo/Redo

        if initial_state:
            self.push_state(initial_state)

    def push_state(self, state: Union[UMLDiagram, dict]) -> None:

```

```

if self._is_restoring:
    return

if isinstance(state, dict):
    try:
        state = UMLDiagram.model_validate(state)
    except Exception as e:
        logger.error(f"History: Validation error: {e}")
    return

is_dup, reason = self._check_duplicate(state)
if is_dup:
    logger.debug(f"History: Push skipped. Reason: {reason}")
    return

new_snapshot = state.model_copy(deep=True)

with self._lock:
    if self._current_index < len(self._stack) - 1:
        logger.info(f"History: Redo branch pruned ({len(self._stack) - 1 - self._current_index} states)")
        self._stack = self._stack[:self._current_index + 1]

    self._stack.append(new_snapshot)
    self._current_index += 1

    # Ограничение размера стека
    if len(self._stack) > self._max_history:
        self._stack.pop(0)
        self._current_index -= 1

    logger.info(f"History: State #{self._current_index} saved. Stack size: {len(self._stack)}")
    self._emit_status()

def undo(self) -> Optional[UMLDiagram]:
    with self._lock:
        if not self.can_undo():
            logger.debug("History: Undo not available")
            return None

        self._is_restoring = True
        try:
            self._current_index -= 1
            state = self._stack[self._current_index].model_copy(deep=True)
        finally:
            self._is_restoring = False

        logger.info(f"History: Undo performed. New index: {self._current_index}")
        self.state_restored.emit(state)
        self._emit_status()
    return state

def redo(self) -> Optional[UMLDiagram]:
    with self._lock:
        if not self.can_redo():
            logger.debug("History: Redo not available")
            return None

        self._is_restoring = True
        try:
            self._current_index += 1
            state = self._stack[self._current_index].model_copy(deep=True)
        finally:
            self._is_restoring = False

        logger.info(f"History: Redo performed. New index: {self._current_index}")
        self.state_restored.emit(state)

```

```

self._emit_status()
return state

def can_undo(self) -> bool:
    return self._current_index > 0

def can_redo(self) -> bool:
    return self._current_index < len(self._stack) - 1
src/logiccraft/services/history_service.py

```

6.4. Копирование, вставка, дублирование

Для реализации буфера обмена (Clipboard) внутри контроллера заведен внутренний регистр `_clipboard: List[dict]`. Так как копирование объектов графа подразумевает, что при вставке они должны получить новые уникальные `UUID`, чтобы не конфликтовать со своими прототипами, мы сериализуем копируемые узлы в dict-формат Pydantic, сбрасывая исходные ID. Рассмотрим реализацию методов буфера обмена в `DiagramController`:

```

def _on_state_restored(self, diagram: UMLDiagram):
    self.manager.diagram = diagram.model_copy(deep=True)
    self.card_map.clear()
    self.connection_map.clear()
    self.diagram_loaded.emit()

def _on_history_changed(self, cu, cr):
    self.undo_redo_changed.emit()

def register_card_view(self, nid, w): self.card_map[nid] = w
def register_connection_view(self, cid, w): self.connection_map[cid] = w

def clear_diagram(self):
    self.manager.clear()
    self.card_map.clear()
    self.connection_map.clear()
    self.diagram_cleared.emit()
    self._save_state()

def duplicate_card(self, card_id: str, offset_x: float = 30, offset_y: float = 30) -> Optional[UMLNode]:
    try:
        source = self.manager.get_node_by_id(card_id)
        if not source:
            return None
        node = self.manager.add_node(
            source.x + offset_x,
            source.y + offset_y,
            name=source.name + "_copy",
            node_type=source.node_type
        )
        from ..models.diagram import UMLProperty, UMLMethod
        node.properties = [p.model_copy() for p in source.properties]
        node.methods = [m.model_copy() for m in source.methods]
        node.enum_literals = [l.model_copy() for l in source.enum_literals]
        node.is_abstract = source.is_abstract
        self.card_added.emit(node)
        self._save_state()
        self.status_changed.emit(f"Класс {node.name} дублирован")
        return node
    except Exception as e:
        self.error_occurred.emit(f"Ошибка дублирования: {e}")
        return None

def copy_selected(self, card_ids: list,
                  actor_ids: list = None,

```

scenario_ids: list = None) -> None:

self._clipboard = []

self._clipboard_uc_actors = []

self._clipboard_uc_scenarios = []

for card_id **in** card_ids:

node = self.manager.get_node_by_id(card_id)

if node:

self._clipboard.append(node.model_copy(deep=True))

for actor_id **in** (actor_ids **or** []):

actor = next((a **for** a **in** self.manager.diagram.uc_actors **if** a.id == actor_id), None)

if actor:

self._clipboard_uc_actors.append(actor.model_copy(deep=True))

for scenario_id **in** (scenario_ids **or** []):

scenario = next((s **for** s **in** self.manager.diagram.uc_scenarios **if** s.id == scenario_id), None)

if scenario:

self._clipboard_uc_scenarios.append(scenario.model_copy(deep=True))

def paste_clipboard(self) -> list:

new_nodes = []

Вставка обычных классов

for source **in** getattr(self, '_clipboard', []):

node = self.manager.add_node(

source.x + 40,

source.y + 40,

name=source.name + "_copy",

node_type=source.node_type

)

node.properties = [p.model_copy() **for** p **in** source.properties]

node.methods = [m.model_copy() **for** m **in** source.methods]

node.enum_literals = [l.model_copy() **for** l **in** source.enum_literals]

node.is_abstract = source.is_abstract

self.card_added.emit(node)

new_nodes.append(node)

Вставка UC-актёров

for source **in** getattr(self, '_clipboard_uc_actors', []):

actor = self.add_uc_actor(source.x + 40, source.y + 40,

name=source.name + "_copy")

if actor:

new_nodes.append(actor)

Вставка UC-сценариев

for source **in** getattr(self, '_clipboard_uc_scenarios', []):

scenario = self.add_uc_scenario(source.x + 40, source.y + 40,

name=source.name + "_copy")

if scenario:

new_nodes.append(scenario)

if new_nodes:

self._save_state()

self.status_changed.emit(f"Вставлено {len(new_nodes)} элементов")

return new_nodes

def select_all(self) -> list:

"""Вернуть все ID карточек для выделения"""

return [node.id **for** node **in** self.manager.diagram.nodes]

src/logiccraft/controllers/diagram_controller.py

6.5. История изменений

Когда пользователь нажимает кнопку «Undo», `HistoryService` достает из недр стека предыдущий слепок `UMLDiagram` и бросает сигнал `state_restored`. Контроллер ловит этот сигнал внутри слота `_on_state_restored`. Задача этого метода — полностью подменить корневой объект диаграммы в `DiagramManager` и принудительно очистить и перерисовать всю графическую сцену (View) с нуля, чтобы UI точно соответствовал откатанной модели.

```
def _on_state_restored(self, diagram: UMLDiagram):
    self.manager.diagram = diagram.model_copy(deep=True)
    self.card_map.clear()
    self.connection_map.clear()
    self.diagram_loaded.emit()
src/logiccraft/controllers/diagram_controller.py
```

Функция эквивалентности `push_state` внутри `HistoryService` проверяет, изменилось ли что-то существенное в структуре, отсекая шум от мелкого дрожания мыши пользователя при кликах:

```
def push_state(self, state: Union[UMLDiagram, dict]) -> None:
    if self._is_restoring:
        return
    if isinstance(state, dict):
        try:
            state = UMLDiagram.model_validate(state)
        except Exception as e:
            logger.error(f"History: Validation error: {e}")
            return
    is_dup, reason = self._check_duplicate(state)
    if is_dup:
        logger.debug(f"History: Push skipped. Reason: {reason}")
        return
    new_snapshot = state.model_copy(deep=True)
    with self._lock:
        if self._current_index < len(self._stack) - 1:
            logger.info(f"History: Redo branch pruned ({len(self._stack) - 1 - self._current_index} states)")
            self._stack = self._stack[:self._current_index + 1]
        self._stack.append(new_snapshot)
        self._current_index += 1
        if len(self._stack) > self._max_history:
            self._stack.pop(0)
            self._current_index -= 1

    logger.info(f"History: State #{self._current_index} saved. Stack size: {len(self._stack)}")
    self._emit_status()
src/logiccraft/services/history_service.py
```

6.6. Валидация диаграммы и обработка ошибок

Валидация и контроль ошибок в архитектуре *LogicCraft* происходят в два этапа. Первый этап работает на уровне сервиса истории — это интеллектуальный метод `_check_duplicate`. Он вызывается непосредственно в `HistoryService` при попытке сделать `push_state`. Его главная задача — покомпонентно сопоставить новое состояние с предыдущим. Если изменения коснулись лишь микро-сдвигов мыши в пределах половины пикселя («мертвая зона»), метод заблокирует запись дубликата, предотвращая переполнение оперативной памяти идентичными слепками.

Рассмотрим внутреннюю реализацию этого алгоритма:

```
def _check_duplicate(self, state: UMLDiagram) -> Tuple[bool, str]:
    if self._current_index == -1:
```

```

    return False, "Initial push"
current = self._stack[self._current_index]
if len(state.nodes) != len(current.nodes) or \
    len(state.connections) != len(current.connections) or \
    len(state.uc_actors) != len(current.uc_actors) or \
    len(state.uc_scenarios) != len(current.uc_scenarios) or \
    len(state.uc_connections) != len(current.uc_connections):
    return False, "Structure changed"
curr_nodes = {n.id: n for n in current.nodes}
for node in state.nodes:
    cn = curr_nodes.get(node.id)
    if not cn: return False, "Node ID changed"
    if abs(node.x - cn.x) > 0.5 or abs(node.y - cn.y) > 0.5:
        return False, f"Movement: {node.name} moved significantly"
    if node.name != cn.name or node.properties != cn.properties:
        return False, "Data inside node changed"
# Проверка связей
curr_conns = {c.id: c for c in current.connections}
for conn in state.connections:
    cc = curr_conns.get(conn.id)
    if not cc or conn.type != cc.type or \
        conn.source_anchor != cc.source_anchor:
        return False, "Connection attributes changed"

```

src/logiccraft/services/history_service.py

Второй этап — это содержательный семантический анализ бизнес-правил UML перед отправкой проекта в кодогенератор, реализованный внутри DiagramController. Он проверяет модель на дублирование классов и нарушение правил связей (например, связи Include и Extend по стандарту UML должны объединять исключительно прецеденты):

```

def validate_diagram(self) -> list:
    warnings = []
    diagram = self.manager.diagram
    # Дублирующиеся имена классов
    names = [n.name for n in diagram.nodes]
    duplicates = {n for n in names if names.count(n) > 1}
    for name in duplicates:
        warnings.append(f"⚠ Дублирующееся имя класса: «{name}»")
    # Классы без атрибутов и методов
    for node in diagram.nodes:
        if not node.properties and not node.methods and node.node_type.value == 'class':
            warnings.append(f"⚠ Класс «{node.name}» не имеет атрибутов и методов")
    # Интерфейс без методов
    for node in diagram.nodes:
        if node.node_type.value == 'interface' and not node.methods:
            warnings.append(f"⚠ Интерфейс «{node.name}» не имеет методов")
    # Циклические зависимости наследования
    inheritance = {}
    for node in diagram.nodes:
        inheritance[node.id] = []
    for conn in diagram.connections:
        if conn.type.value == 'inheritance':
            inheritance[conn.source_id].append(conn.target_id)
    def has_cycle(node_id, visited, path):
        if node_id in path:
            return True

```

```

if node_id in visited:
    return False
visited.add(node_id)
path.add(node_id)
for child in inheritance.get(node_id, []):
    if has_cycle(child, visited, path):
        return True
path.discard(node_id)
return False

visited = set()
for node in diagram.nodes:
    if has_cycle(node.id, visited, set()):
        warnings.append(f"❌ Циклическое наследование обнаружено в «{node.name}»")
        break

# Связи к несуществующим узлам
node_ids = {n.id for n in diagram.nodes}
for conn in diagram.connections:
    if conn.source_id not in node_ids or conn.target_id not in node_ids:
        warnings.append(f"❌ Связь ссылается на несуществующий класс")

```

src/logiccraft/controllers/diagram_controller.py

Глава 7. Генерация кода

Одной из ключевых возможностей интегрированных сред проектирования и UML-архитекторов является автоматическая трансляция графических моделей в текстовое представление исходного кода на целевых языках программирования. В приложении **LogicCraft** этот функционал отделен от графического интерфейса и сосредоточен в изолированном слое сервисов. Такой подход обеспечивает гибкость, независимость бизнес-логики от UI и позволяет легко масштабировать систему, добавляя поддержку новых языков программирования без изменения существующих подсистем.

В этой главе мы подробно разберем, как спроектировать и реализовать гибкий генератор кода, использующий популярный шаблонизатор **Jinja2**, разберем устройство конфигурационных файлов, тонкости синтаксиса шаблонов для объектно-ориентированных языков, алгоритмы обхода графа связей и особенности физической записи результатов на диск.

7.1. Архитектура генератора кода

В основе архитектуры генерации кода LogicCraft лежит сочетание классического объектного подхода и паттерна распределенной конфигурации. Задачу трансформации UML-модели в текст решает класс **CodeGenerator**. Основная идея его проектирования заключается в том, что сам класс генератора не содержит зашитаго синтаксиса конкретных языков программирования. Вместо этого он выступает в роли координатора: считывает метаданные из конфигурационных файлов, инициализирует подсистему шаблонизации, подготавливает внутреннее представление объектного графа и передает его движку рендеринга.

Ниже представлена полная программная реализация класса **CodeGenerator**, находящаяся в модуле **src/logiccraft/services/code_generator.py**. В конструкторе класса настраивается среда окружения Jinja2 с поддержкой кастомных путей, что позволяет пользователям расширять возможности приложения без вмешательства в его дистрибутив.

```

from typing import List, Dict, Optional
import os
from pathlib import Path
from jinja2 import Environment, FileSystemLoader, Template
from ..models.diagram import UMLDiagram, UMLNode, UMLProperty, UMLMethod, NodeType
from ..templates.config import get_language_config, get_supported_languages, map_type

class CodeGenerator:
    def __init__(self):

```



```

self.language = "python"
# Настройка Jinja2 окружения для встроенных шаблонов
templates_dir = Path(__file__).parent.parent / "templates"
# Добавляем путь к пользовательским шаблонам
user_templates_dir = Path.home() / ".logiccraft" / "templates"
user_templates_dir.mkdir(parents=True, exist_ok=True)
# Создаём загрузчик с несколькими путями (сначала пользовательские, потом встроенные)
from jinja2 import ChoiceLoader, FileSystemLoader
self.env = Environment(
    loader=ChoiceLoader([
        FileSystemLoader(user_templates_dir),
        FileSystemLoader(templates_dir)
    ]),
    trim_blocks=True,
    lstrip_blocks=True
)

self.env.globals.update({
    'get_python_visibility': self._get_python_visibility,
    'get_java_visibility': self._get_java_visibility,
    'generate_method_params': self._generate_method_params,
})

def generate(self, diagram: UMLDiagram, language: str = "python", custom_template: str = None) -> str:
    if language not in get_supported_languages():
        raise ValueError(f"Unsupported language: {language}. Supported: {get_supported_languages()}")

    self.language = language
    config = get_language_config(language)
    if custom_template:
        try:
            template = self.env.get_template(f"{custom_template}.j2")
        except Exception:
            # Если не найден, используем стандартный
            template = self.env.get_template(config["template"])
    else:
        template = self.env.get_template(config["template"])
    # Создаем карту наследования
    inheritance_map = self._build_inheritance_map(diagram)

    # Проверяем типы узлов для импортов
    has_abstract_classes = any(node.is_abstract or node.node_type.value == 'abstract_class' for node in diagram.nodes)
    has_interfaces = any(node.node_type.value == 'interface' for node in diagram.nodes)
    has_enums = any(node.node_type.value == 'enum' for node in diagram.nodes)

    return template.render(
        diagram_name=diagram.name,
        nodes=diagram.nodes,
        inheritance_map=inheritance_map,
        has_abstract_classes=has_abstract_classes,
        has_interfaces=has_interfaces,
        has_enums=has_enums,
        language=language
    )

def generate_files(self, diagram: UMLDiagram, language: str = "python") -> Dict[str, str]:

```

```

"""Сгенерировать файлы для диаграммы (по одному файлу на класс)"""
if language not in get_supported_languages():
    raise ValueError(f"Unsupported language: {language}. Supported: {get_supported_languages()}")

files = {}
config = get_language_config(language)

for node in diagram.nodes:
    filename = f"{node.name}{config['extension']}"
    content = self._generate_single_class(node, diagram, language)
    files[filename] = content

return files

def get_supported_languages(self) -> List[str]:
    """Получить список поддерживаемых языков"""
    return get_supported_languages()

def _get_filename(self, class_name: str, language: str) -> str:
    """Получить имя файла для класса"""
    config = get_language_config(language)
    return f"{class_name}{config['extension']}"

def _generate_single_class(self, node: UMLNode, diagram: UMLDiagram, language: str) -> str:
    """Сгенерировать код для одного класса"""
    config = get_language_config(language)
    template = self.env.get_template(config["template"])

    # Создаем карту наследования
    inheritance_map = self._build_inheritance_map(diagram)

    # Проверяем, есть ли абстрактные классы (для Python)
    has_abstract_classes = node.is_abstract

    return template.render(
        diagram_name=diagram.name,
        nodes=[node],
        inheritance_map=inheritance_map,
        has_abstract_classes=has_abstract_classes,
        language=language
    )

def _build_inheritance_map(self, diagram: UMLDiagram) -> Dict[str, List[str]]:
    """Построить карту наследования из связей диаграммы"""
    inheritance_map = {}

    # Инициализируем пустые списки для всех узлов
    for node in diagram.nodes:
        inheritance_map[node.name] = []

    # Заполняем карту на основе связей (если они есть)
    if hasattr(diagram, 'connections'):
        for connection in diagram.connections:
            if connection.type.value == "inheritance":
                # Находим имена узлов по ID
                source_node = next((n for n in diagram.nodes if n.id == connection.source_id), None)
                target_node = next((n for n in diagram.nodes if n.id == connection.target_id), None)

                if source_node and target_node:
                    inheritance_map[source_node.name].append(target_node.name)

    return inheritance_map

```

```

def _generate_method_params(self, method: UMLMethod) -> str:
    """Генерация параметров метода"""
    params = []
    for param in method.parameters:
        params.append(f'{param.name}: {param.type}')
    return ", ".join(params)

def _get_python_visibility(self, visibility: str) -> str:
    """Преобразовать видимость в Python"""
    if visibility == "private":
        return "_"
    elif visibility == "protected":
        return "_"
    else: # public
        return ""

def _get_java_visibility(self, visibility: str) -> str:
    """Преобразовать видимость в Java"""
    if visibility == "private":
        return "private"
    elif visibility == "protected":
        return "protected"
    else: # public
        return "public"

```

7.2. Промежуточная модель для генерации

Перед непосредственной отправкой объектного графа диаграммы в движок Jinja2, генератор выполняет предварительный синтаксический анализ структуры данных в методе `generate`. Этот этап критически важен для оптимизации итогового кода и формирования правильных метаданных (контекста), которые ожидает шаблон.

Вместо сложной трансформации самих объектов узлов, `CodeGenerator` использует предикативный анализ с помощью генераторов списков и функции `any()`. Это позволяет динамически вычислять глобальные свойства диаграммы до начала рендеринга текста:

```

has_abstract_classes = any(node.is_abstract or node.node_type.value == 'abstract_class' for node in diagram.nodes)
has_interfaces = any(node.node_type.value == 'interface' for node in diagram.nodes)
has_enums = any(node.node_type.value == 'enum' for node in diagram.nodes)

```

Анализ структуры решает следующие задачи:

- Оптимизация пространства импортов:** Модули верхнего уровня шаблона получают сигналы о необходимости включения специализированных библиотек. Если `has_abstract_classes` или `has_interfaces` истинны, в заголовок файла Python будут добавлены инструкции `from abc import ABC, abstractmethod`. Если истинен флаг `has_enums`, подключится базовый класс `from enum import Enum`.
- Изоляция исходных данных:** Сама структура `diagram.nodes` передается в шаблон в исходном виде, что исключает появление побочных эффектов (side-effects) и случайную модификацию внутренних моделей данных GUI-редактора во время выполнения операции экспорта.

7.3. Шаблоны Jinja2

Для полной изоляции логики разметки исходного кода от управляющих алгоритмов Python применяется механизм шаблонизации **Jinja2**. Он позволяет использовать декларативное описание структуры файлов и управлять разметкой кода с помощью управляющих инструкций, циклов и встроенных макросов.

Для управления окружением Jinja2 в конструкторе `CodeGenerator` применяется класс `ChoiceLoader`. Это дает нашему архитектору мощную точку расширения: приложение сначала проверяет наличие специализированного файла шаблона в домашней директории пользователя `~/.logiccraft/templates/`, и если пользователь создал там кастомный файл, приложение приоритетно использует его. Если кастомный шаблон отсутствует, загрузчик автоматически откатывается на встроенные ресурсы приложения.

Параметры окружения `trim_blocks=True` и `lstrip_blocks=True` имеют решающее значение при генерации исходных текстов программ. Без этих параметров пустые строки управления Jinja2 (такие как `{% if %}` и `{% endif %}`) оставляли бы после себя пустые строки и некорректные сдвиги табуляции, нарушая требования стандартов оформления кода (PEP 8 в Python или Java Code Conventions).

Дополнительно в окружение Jinja2 через `self.env.globals.update` внедряются специализированные Python-функции обратного вызова (helpers): `get_python_visibility`, `get_java_visibility` и `generate_method_params`. Это позволяет вынести громоздкую логику конкатенации строк и разбора параметров методов из текстового шаблона обратно в Python-код.

7.4. Генерация Python-кода

При генерации исходного кода на языке Python шаблонизатор должен учитывать специфические особенности среды: отсутствие явных модификаторов доступа (использование нижних подчеркиваний), динамическую природу с опциональной типизацией и явную передачу ссылки на объект (`self`) в качестве первого параметра методов экземпляра класса.

Поведение генератора управляется конфигурационным модулем `src/logiccraft/templates/config.py`. Для языка Python в нем определены правила маппинга типов и подстановки дефолтных значений:

```
LANGUAGE_CONFIGS = {
    "python": {
        "extension": ".py",
        "template": "python_class.j2",
        "type_mappings": {
            "string": "str",
            "integer": "int",
            "boolean": "bool",
            "float": "float",
            "list": "List",
            "dict": "Dict"
        },
        "default_values": {
            "str": "",
            "int": "0",
            "bool": "False",
            "float": "0.0",
            "List": "[]",
            "Dict": "{}"
        }
    },
}
```

Таблицы подстановок типов (`type_mappings`) преобразуют абстрактные типы метамодели диаграммы в лексические типы данных Python (например, перевод типа `string` в `str`). Внедренный хелпер `get_python_visibility` транслирует строковые значения уровней доступа UML (`private`, `protected`, `public`) в префиксы стандартов Python: `__` (double underscore) для приватных атрибутов и `_` (single underscore) для защищенных.

Ниже представлен исходный код шаблона `src/logiccraft/templates/python_class.j2`, обрабатывающий эти правила:

```
"""Generated from UML Diagram: {{ diagram_name }}"""
from typing import List, Optional
{% if has_abstract_classes or has_interfaces %}
from abc import ABC, abstractmethod
{% endif %}
{% if has_enums %}
from enum import Enum
{% endif %}

{% for node in nodes %}
{% set ntype = node.node_type.value %}
{% if ntype == 'interface' %}
class {{ node.name }}(ABC):
    """Interface: {{ node.name }}"""
{% elif ntype == 'enum' %}
```

```

class {{ node.name }}(Enum):
    """Enumeration: {{ node.name }}"""
{% if node.enum_literals %}
{% for literal in node.enum_literals %}
    {{ literal.name }}{% if literal.value %} = {{ literal.value }}{% else %} = {{ loop.index }}{% endif %}

{% endfor %}
{% endif %}
{% elif ntype == 'abstract_class' or node.is_abstract %}
class {{ node.name }}(ABC{% if inheritance_map[node.name] %}, {{ inheritance_map[node.name] | join(',') }}{% endif %}):
    """Abstract class: {{ node.name }}"""
{% else %}
class {{ node.name }}{% if inheritance_map[node.name] %}({{ inheritance_map[node.name] | join(',') }}){% endif %}:
    """Class generated from UML diagram"""
{% endif %}
{% if ntype != 'enum' %}
{% if node.properties %}

    # Attributes
{% for prop in node.properties %}
    {{ get_python_visibility(prop.visibility) }}{{ prop.name }}: {{ prop.type }}{% if prop.default_value %} = {{ prop.default_value }}{%
endif %}

{% endfor %}
{% endif %}
{% if ntype != 'interface' %}

    def __init__(self):
{% if node.properties %}
{% for prop in node.properties %}
        {% if prop.default_value %}
            self.{{ prop.name }} = {{ prop.default_value }}
        {% else %}
            self.{{ prop.name }} = None
        {% endif %}
{% endfor %}
{% else %}
        pass
{% endif %}
{% endif %}
{% if node.methods %}

    # Methods
{% for method in node.methods %}
{% if method.is_abstract or ntype == 'interface' %}
        @abstractmethod
{% elif method.is_static %}
        @staticmethod
{% endif %}
        {% if method.is_static %}
            def {{ method.name }}({{ generate_method_params(method) }}){% if method.return_type %} -> {{ method.return_type }}{%
endif %}:
        {% else %}
            def {{ method.name }}(self{% if generate_method_params(method) %}, {{ generate_method_params(method) }}{% endif %}){%
if method.return_type %} -> {{ method.return_type }}{% else %} -> None{% endif %}:
        {% endif %}
        {% if method.is_abstract or ntype == 'interface' %}
            raise NotImplementedError

```

```

    {% else %}
    pass
    {% endif %}
{% if not loop.last %}

{% endif %}
{% endfor %}
{% endif %}
{% endif %}
{% if not loop.last %}

```

```

{% endif %}
{% endfor %}

```

7.5. Генерация Java-кода

Проектирование генератора для объектно-ориентированных языков со строгой статической типизацией, таких как Java, накладывает на подсистему генерации кода ряд дополнительных требований, принципиально отличающихся от динамического Python. В экосистеме Java каждый файл должен представлять собой законченную синтаксическую единицу, а типы данных, модификаторы доступа и сигнатуры методов должны быть жестко определены на этапе компиляции. Для управления генерацией Java-кода в модуле конфигурации `src/logiccraft/templates/config.py` задана специальная таблица маппинга. Она преобразует абстрактные типы метамодели UML в строго типизированные эквиваленты платформы Java (например, концептуальный тип `list` инициализируется как полноценная коллекция, а `string` — как объект `String`):

```

LANGUAGE_CONFIGS = {
    "java": {
        "extension": ".java",
        "template": "java_class.j2",
        "type_mappings": {
            "string": "String",
            "integer": "int",
            "boolean": "boolean",
            "float": "double",
            "list": "List<Object>",
            "dict": "Map<String, Object>"
        },
        "default_values": {
            "String": "",
            "int": "0",
            "boolean": "false",
            "double": "0.0",
            "List<Object>": "new ArrayList<>()",
            "Map<String, Object>": "new HashMap<>()"
        }
    }
}

```

Для сопоставления модификаторов видимости `CodeGenerator` использует внутренний хелпер `_get_java_visibility`, который переводит строковые значения UML-модели в явные ключевые слова Java (`public`, `protected`, `private`). Если в UML-свойстве или методе взведен флаг `is_static`, генератор добавляет модификатор `static`.

Ниже представлен полный исходный код шаблона `src/logiccraft/templates/java_class.j2`, реализующий логику трансформации для классов, абстрактных классов, интерфейсов и перечислений (`enum`):

```

{% for node in nodes %}
{% set ntype = node.node_type.value %}
{% if ntype == 'interface' %}
public interface {{ node.name }}{% if inheritance_map[node.name] %} extends {{ inheritance_map[node.name] | join(', ') }}{%
endif %} {

```

```

{% elif ntype == 'enum' %}
public enum {{ node.name }} {
{% if node.enum_literals %}
{% for literal in node.enum_literals %}
    {{ literal.name }}{% if not loop.last %},{% else %};{% endif %}

{% endfor %}
{% endif %}
{% elif ntype == 'abstract_class' or node.is_abstract %}
public abstract class {{ node.name }}{% if inheritance_map[node.name] %} extends {{ inheritance_map[node.name][0] }}{%
endif %} {
{% else %}
public class {{ node.name }}{% if inheritance_map[node.name] %} extends {{ inheritance_map[node.name][0] }}{% endif %} {
{% endif %}
{% if ntype != 'enum' %}
{% if node.properties %}

    // Attributes
{% for prop in node.properties %}
    {{ get_java_visibility(prop.visibility) }}{% if prop.is_static %} static{% endif %} {{ prop.type }} {{ prop.name }};
{% endfor %}
{% endif %}

    // Constructor
    public {{ node.name }}() {
        // TODO: Initialize attributes
    }
{% endif %}
{% if node.methods %}

    // Methods
{% for method in node.methods %}
    {{ get_java_visibility(method.visibility) }}{% if method.is_abstract or ntype == 'interface' %} abstract{% endif %}{% if
method.is_static %} static{% endif %} {{ method.return_type or "void" }} {{ method.name }}{% for param in
method.parameters %}{{ param.type }} {{ param.name }}{% if not loop.last %}, {% endif %}{% endfor %}}{% if method.is_abstract or
ntype == 'interface' %};{% else %} {
        // TODO: Implement method
    }{% endif %}
{% if not loop.last %}

{% endif %}
{% endfor %}
{% endif %}
{% endif %}

```

Анатомия этого шаблона учитывает, что интерфейсы в Java не могут иметь конструкторов и стандартных полей экземпляра, а их методы по умолчанию являются абстрактными и не должны содержать тела (блока {{}}). Перечисления (enum) обрабатываются через итерацию списка литералов node.enum_literals, где элементы перечисляются через запятую, а последний завершается точкой с запятой.

7.6. Обработка наследования и связей

UML-диаграмма — это не просто набор изолированных сущностей, а связный ориентированный граф, в котором ребра определяют отношения между классами. Наиболее критичным отношением для генерации кода является наследование (обобщение/генерализация). Ошибки при его обработке могут привести к потере иерархии или генерации синтаксически неверных конструкций.

Внутри класса `CodeGenerator` за синтаксический разбор топологии связей диаграммы отвечает приватный метод `_build_inheritance_map`. Его задача — проанализировать массив соединений `connections` и построить ассоциативный словарь, где ключом является имя дочернего класса, а значением — список имен его непосредственных предков:

```
def _build_inheritance_map(self, diagram: UMLDiagram) -> Dict[str, List[str]]:
    """Построить карту наследования из связей диаграммы"""
    inheritance_map = {}

    # Инициализируем пустые списки для всех узлов
    for node in diagram.nodes:
        inheritance_map[node.name] = []

    # Заполняем карту на основе связей (если они есть)
    if hasattr(diagram, 'connections'):
        for connection in diagram.connections:
            if connection.type.value == "inheritance":
                # Находим имена узлов по ID
                source_node = next((n for n in diagram.nodes if n.id == connection.source_id), None)
                target_node = next((n for n in diagram.nodes if n.id == connection.target_id), None)

                if source_node and target_node:
                    inheritance_map[source_node.name].append(target_node.name)

    return inheritance_map
```

Этот алгоритм работает в два этапа:

1. **Инициализация:** Для каждого узла диаграммы в словаре резервируется пустой список. Это гарантирует, что даже если у класса нет родителей, обращение к `inheritance_map[node.name]` в шаблоне не вызовет исключения `KeyError`.
2. **Сканирование ребер:** Программа фильтрует соединения по типу `connection.type.value == "inheritance"`. По идентификаторам `source_id` (источник — дочерний элемент) и `target_id` (цель — родительский элемент) находятся соответствующие объекты узлов, и имя родителя добавляется в список.

Полученная карта передается напрямую в контекст рендеринга Jinja2. Внутри шаблонов логика ветвления адаптируется под специфику конкретного языка программирования:

- В шаблоне **Python** используется выражение `{{ inheritance_map[node.name] | join(' ', ' ') }}`, что позволяет генерировать множественное наследование, перечисляя базовые классы через запятую внутри круглых скобок: `class Child(Parent1, Parent2):`.
- В шаблоне **Java** множественное наследование классов запрещено спецификацией языка. Поэтому шаблон принудительно извлекает только первый элемент массива: `extends {{ inheritance_map[node.name][0] }}`. Это защищает итоговый код от синтаксических ошибок, если пользователь ошибочно провел две линии наследования к двум разным классам.

7.7. Генерация пакетов и модулей

На уровне базовой генерации в один строковый буфер (метод `generate`) приложение объединяет все классы в рамках единого текстового пространства. Однако для создания масштабируемой архитектуры приложения требуется распределение сущностей по физическим пакетам и модулям на жестком диске.

В LogicCraft за этот функционал отвечает метод `generate_files`. Вместо компиляции единого монолитного текста, он осуществляет декомпозицию диаграммы, подготавливая структуру данных для создания полноценного физического пакета:

```
def generate_files(self, diagram: UMLDiagram, language: str = "python") -> Dict[str, str]:
    """Сгенерировать файлы для диаграммы (по одному файлу на класс)"""
    if language not in get_supported_languages():
        raise ValueError(f"Unsupported language: {language}. Supported: {get_supported_languages()}")
```



```

files = {}
config = get_language_config(language)

for node in diagram.nodes:
    filename = f"{node.name}{config['extension']}"
    content = self._generate_single_class(node, diagram, language)
    files[filename] = content

return files

```

При генерации модулей учитываются языковые стандарты целевых платформ:

1. **Пакеты Python:** При итерации по узлам каждый класс выносится в отдельный модуль `.py`. Для превращения директории экспорта в легитимный пакет Python (Package Scope) верхнеуровневый контроллер экспорта автоматически дописывает пустой файл инициализации `__init__.py`. Это позволяет в дальнейшем импортировать сгенерированные классы стандартным путем (`from .ClassName import ClassName`).
2. **Пакеты Java:** Имя файла строго привязывается к имени класса (`node.name + .java`). Генерация изолированных файлов для каждого узла здесь является обязательным требованием компилятора `javac`. В последующих архитектурных надстройках иерархия вложенности пакетов на диаграмме (с использованием пространств имен) напрямую транслируется в физическое дерево папок на диске.

7.8. Пользовательские шаблоны

Профессиональные разработчики часто сталкиваются с необходимостью кастомизации генерируемого кода под внутренние стандарты компании, специфичные фреймворки (например, Django ORM или Spring Boot) или личные предпочтения. Для решения этой задачи в `CodeGenerator` заложена гибкая двухкомпонентная система поддержки пользовательских шаблонов.

Первый уровень кастомизации реализуется на этапе инициализации генератора в конструкторе через встроенный механизм `ChoiceLoader` библиотеки Jinja2:

```

# Добавляем путь к пользовательским шаблонам
user_templates_dir = Path.home() / ".logiccraft" / "templates"
user_templates_dir.mkdir(parents=True, exist_ok=True)
# Создаём загрузчик с несколькими путями (сначала пользовательские, потом встроенные)
self.env = Environment(
    loader=ChoiceLoader([
        FileSystemLoader(user_templates_dir),
        FileSystemLoader(templates_dir)
    ]),
    trim_blocks=True,
    lstrip_blocks=True
)

```

Загрузчик `ChoiceLoader` принимает список объектов `FileSystemLoader`. При запросе шаблона система сначала обращается к изолированной пользовательской директории в домашнем каталоге операционной системы (`~/logiccraft/templates/`). Если пользователь разместил там файл с системным именем (например, `python_class.j2`), приложение приоритетно загрузит его, позволяя полностью изменить структуру генерации без модификации самого исходного кода LogicCraft. Если файл отсутствует, происходит бесшовный откат (fallback) к стандартным встроенным шаблонам приложения.

Второй уровень кастомизации — это динамическая подмена шаблона на лету с помощью передачи необязательного аргумента `custom_template` напрямую в метод `generate`:

```

# Если указан пользовательский шаблон, пытаемся его загрузить
if custom_template:
    try:
        template = self.env.get_template(f"{custom_template}.j2")
    except Exception:
        # Если не найден, используем стандартный
        template = self.env.get_template(config["template"])

```

Этот механизм обеспечивает атомарность и отказоустойчивость: если переданное имя шаблона (например, `fastapi_route`) не будет обнаружено в путях загрузчика, блок `except` перехватит исключение `TemplateNotFound`, предотвратит падение программы и автоматически выставит дефолтный шаблон, заданный конфигурацией языка.

7.9. Экспорт кода в файлы

В реальной практике проектирования программных систем монолитный экспорт всей UML-диаграммы в один текстовый файл (буфер) применим далеко не всегда. Для языков со строгой файловой структурой (таких как Java) каждый публичный класс, интерфейс или перечисление обязаны находиться в изолированном физическом файле, имя которого строго совпадает с идентификатором сущности. Но даже в языках с более свободной организацией (как Python) разбиение крупных архитектурных схем по отдельным модулям является общепринятым стандартом чистой архитектуры.

Для решения задачи модульного расщепления внутри класса `CodeGenerator` спроектирована подсистема пофайловой генерации, представленная методами `generate_files` и `_generate_single_class`.

Метод `generate_files` выступает в роли диспетчера. Он итерируется по коллекции узлов диаграммы, динамически вычисляет имя будущего файла с нужным расширением через вспомогательный метод `_get_filename` и делегирует рендеринг контента методу одиночной генерации:

```
def generate_files(self, diagram: UMLDiagram, language: str = "python") -> Dict[str, str]:
    """Сгенерировать файлы для диаграммы (по одному файлу на класс)"""
    if language not in get_supported_languages():
        raise ValueError(f"Unsupported language: {language}. Supported: {get_supported_languages()}")

    files = {}
    config = get_language_config(language)
    for node in diagram.nodes:
        filename = f"{node.name}{config['extension']}"
        content = self._generate_single_class(node, diagram, language)
        files[filename] = content
    return files
```

Непосредственное сужение контекста для шаблонизатора происходит внутри метода `_generate_single_class`. Вместо передачи всего списка `diagram.nodes`, в контекст Jinja2 отправляется список, содержащий **только один текущий узел** (`nodes=[node]`):

```
def _generate_single_class(self, node: UMLNode, diagram: UMLDiagram, language: str) -> str:
    """Сгенерировать код для одного класса"""
    config = get_language_config(language)
    template = self.env.get_template(config["template"])
    # Создаем карту наследования
    inheritance_map = self._build_inheritance_map(diagram)
    # Проверяем, есть ли абстрактные классы (для Python)
    has_abstract_classes = node.is_abstract
    return template.render(
        diagram_name=diagram.name,
        nodes=[node],
        inheritance_map=inheritance_map,
        has_abstract_classes=has_abstract_classes,
        language=language
    )
```

Важной архитектурной деталью здесь является то, что граф связей `inheritance_map` по-прежнему строится на основе **всей исходной диаграммы** (`diagram`). Это позволяет изолированному классу при рендеринге сохранять информацию о своих предках и корректно генерировать строки наследования, даже находясь в отдельном файле.

В результате выполнения этого алгоритма подсистема генерации возвращает на верхний уровень ассоциативный массив данных `Dict[str, str]`, где ключ — это имя файла (например, `User.py` или `PaymentService.java`), а значение — готовый текстовый код. Данный словарь абстрагирован от конкретных путей на жестком диске, что позволяет безболезненно передавать его дальше в слой контроллеров (`diagram_controller.py`) для последующего вывода в UI предпросмотра или сохранения в zip-архивы и каталоги операционной системы, о чем подробно пойдет речь в следующей главе.

Глава 8. Экспорт проекта

8.1. Структура готового проекта

При экспорте проекта LogicCraft создает полную файловую структуру, соответствующую выбранному языку программирования и архитектурному шаблону. Экспорт реализуется классом `src/logiccraft/services/project_exporter.py`.

Основные компоненты экспорта

ProjectExporter - основной сервис для создания структуры проекта

ниже представлены не полная версия кода, полную версию можно посмотреть на GitHub(<https://github.com/Trewasasha/LogicCraft>)

class ProjectExporter:

```
def __init__(self):
    self.generator = CodeGenerator()

def export_project(self, diagram: UMLDiagram, settings: ProjectSettings,
    export_path: str) -> Dict[str, Any]:
    project_path = Path(export_path) / settings.name
    structure = self._create_project_structure(settings, project_path)
    files = self.generator.generate_files(diagram, settings.language)
    self._place_code_files(files, structure, settings)
    if settings.include_config:
        self._create_config_files(structure, settings, diagram)
    if settings.include_tests:
        self._create_test_files(structure, settings, diagram)
    if settings.include_docs:
        self._create_documentation(structure, settings, diagram)
    # Создаем файлы на диске
    created_files = self._write_structure_to_disk(structure)
    return {
        "project_path": str(project_path),
        "files_created": len(created_files),
        "structure": structure,
        "settings": settings
    }
```

ProjectSettings - настройки проекта (src/logiccraft/models/project_settings.py)

@dataclass

class ProjectSettings:

```
name: str
language: str
structure_type: str # "simple", "mvc", "layered", "clean_architecture"
framework: Optional[str] = None # "django", "spring", "express", "react"
author: str = ""
description: str = ""
version: str = "1.0.0"
license: Optional[str] = None

package_manager: Optional[str] = None
code_style: CodeStyleSettings = field(default_factory=CodeStyleSettings)

include_tests: bool = False
include_docs: bool = False
include_config: bool = True
include_gitignore: bool = True
include_readme: bool = True
    include_license: bool = False
    include_ci: bool = False

export_path: str = ""
overwrite_existing: bool = False
```

Структура экспорта

```

project_name/
├── README.md           # Документация проекта
├── .gitignore          # Исключения для Git
├── requirements.txt     # Зависимости (Python)
├── setup.py            # Конфигурация пакета (Python)
├── pom.xml             # Maven конфигурация (Java)
├── package.json        # NPM конфигурация (JS/TS)
├── {name}.sln          # Solution файл (C#)
├── src/               # Исходный код
│   └── ...            # В зависимости от языка и структуры
├── tests/             # Тесты (опционально)
├── docs/              # Документация (опционально)
└── .github/           # CI/CD (опционально)

```

8.2. Архитектурные шаблоны: MVC

В отличие от простой структуры, подход **MVC (Model-View-Controller)** предлагает разделение кода на слои по принципу ответственности. Это является фундаментальным шагом при проектировании приложений среднего и большого размера, так как способствует поддержанию порядка в коде, упрощает командную разработку и последующее тестирование.

Структура проекта, организованная по паттерну MVC, выглядит следующим образом:

```

myproject/
├── README.md
├── requirements.txt
├── setup.py
├── .gitignore
└── myproject/
    ├── __init__.py
    ├── models/
    │   └── __init__.py
    ├── views/
    │   └── __init__.py
    ├── controllers/
    │   └── __init__.py
    └── main.py

```

Такое разделение обеспечивает высокую степень модульности, позволяя независимо разрабатывать, тестировать и изменять каждый компонент системы, что критически важно для долгосрочной поддержки и масштабируемости проекта.

8.3. Генерация конфигурационных файлов

Генерация конфигурационных файлов происходит в классе `src/logiccraft/services/project_exporter.py`:

1. Метод `export_project()` - основной метод экспорта, вызывающий:

- `_create_project_structure()` - создает базовую структуру
- `_create_config_files()` - создает дополнительные конфиги (LICENSE, CI)
- `_write_structure_to_disk()` - записывает все на диск

```

def export_project(self, diagram: UMLDiagram, settings: ProjectSettings,
                  export_path: str) -> Dict[str, Any]:
    project_path = Path(export_path) / settings.name
    structure = self._create_project_structure(settings, project_path)
    files = self.generator.generate_files(diagram, settings.language)
    self._place_code_files(files, structure, settings)
    if settings.include_config:
        self._create_config_files(structure, settings, diagram)

```

```

if settings.include_tests:
    self._create_test_files(structure, settings, diagram)
if settings.include_docs:
    self._create_documentation(structure, settings, diagram)
created_files = self._write_structure_to_disk(structure)
return {
    "project_path": str(project_path),
    "files_created": len(created_files),
    "structure": structure,
    "settings": settings
}

```

2. Метод `_create_config_files()` - создает:

- LICENSE - файл лицензии (если `include_license=True`)
- `.github/workflows/ci.yml` - GitHub Actions (если `include_ci=True`)

```

def _create_config_files(self, structure: Dict[str, Any],
                        settings: ProjectSettings, diagram: UMLDiagram):
    children = structure["children"]
    if settings.include_license and settings.license:
        children["LICENSE"] = {"type": "file", "content": self._generate_license(settings)}
    if settings.include_ci:
        children[".github"] = {
            "type": "directory",
            "children": {
                "workflows": {
                    "type": "directory",
                    "children": {
                        "ci.yml": {"type": "file", "content": self._generate_github_actions(settings)}
                    }
                }
            }
        }
    }
}

```

3. Методы генерации контента- создают содержимое файлов:

- `_generate_readme()` - README.md с инструкциями
- `_generate_python_requirements()` - requirements.txt
- `_generate_python_setup()` - setup.py
- `_generate_python_gitignore()` - .gitignore для Python
- `_generate_maven_pom()` - pom.xml для Java
- `_generate_package_json()` - package.json для JS/TS
- `_generate_csharp_solution()` - .sln для C#
- `_generate_license()` - LICENSE файл
- `_generate_github_actions()` - CI workflow

ниже представлены часть кода, полную версию можно посмотреть на GitHub(<https://github.com/Trewasasha/LogicCraft>)

```

def _generate_readme(self, settings: ProjectSettings) -> str:
    author_line = f"\nAuthor: {settings.author}" if settings.author else ""
    desc_line = f"\n{settings.description}" if settings.description else ""
    setup_instructions = self._get_setup_instructions(settings)
    usage_example = self._get_usage_example(settings)

```

Файлы по умолчанию

Все проекты включают базовые конфигурационные файлы:

Файл	Описание
README.md	Документация с инструкциями по установке и использованию

.gitignore	Исключения для системы контроля версий
requirements.txt	Python зависимости (только для Python)
setup.py	Конфигурация пакета (только для Python)
pom.xml	Maven конфигурация (только для Java)
package.json	NPM конфигурация (только для JS/TS)

8.4. Предпросмотр и экспорт на диск

Перед экспортом можно посмотреть структуру:

```
from logiccraft.services.project_exporter import ProjectExporter
from logiccraft.models.project_settings import ProjectSettings
```

```
exporter = ProjectExporter()
settings = ProjectSettings(
    name="MyProject",
    language="python",
    structure_type="mvc"
)
# Получить структуру без записи на диск
structure = exporter._create_project_structure(settings, Path("/tmp"))
```

Экспорт на диск

```
# Полный экспорт проекта
result = exporter.export_project(
    diagram=uml_diagram,
    settings=project_settings,
    export_path="/path/to/export"
)
```

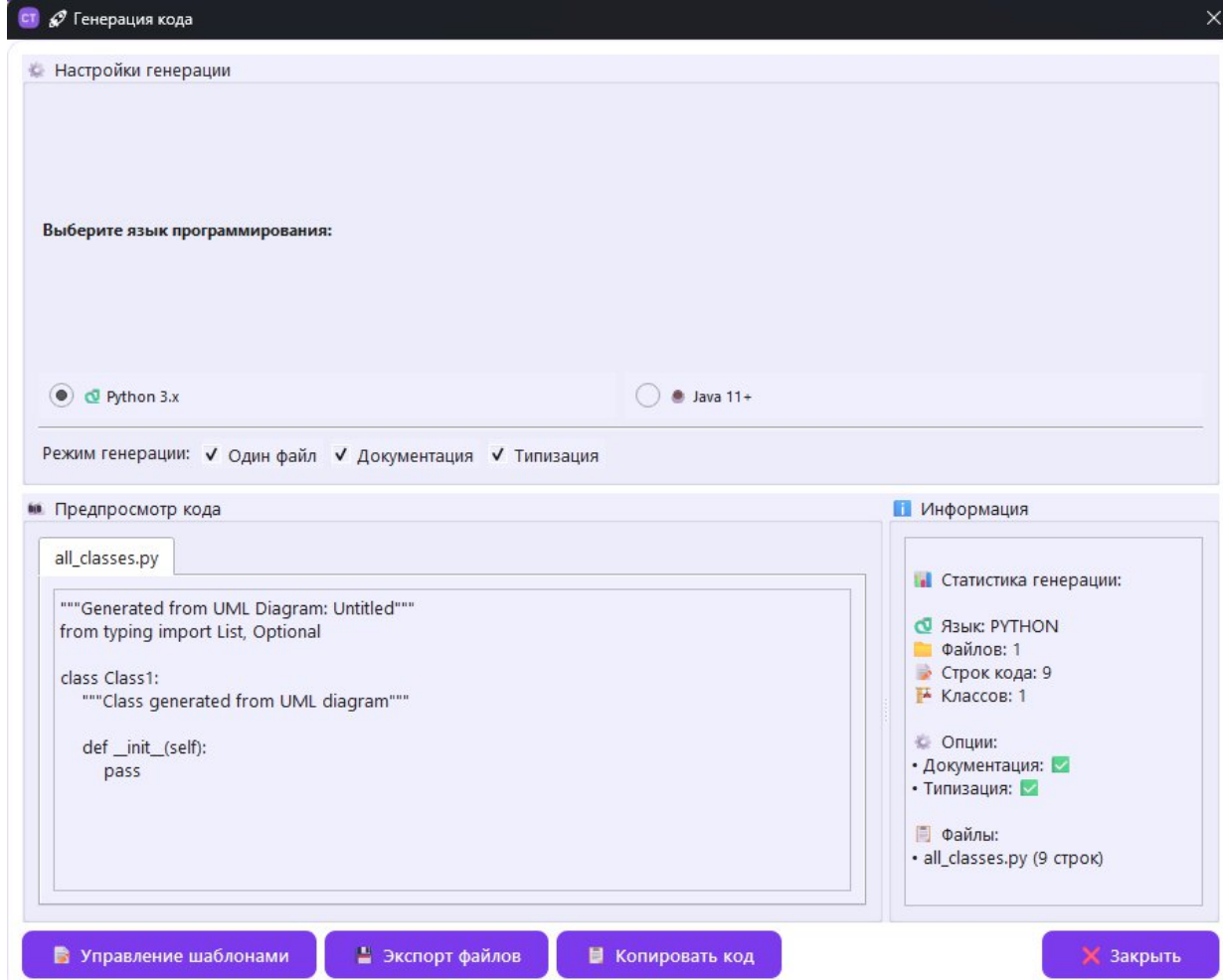
Результат

```
{
    "project_path": "/path/to/export/MyProject",
    "files_created": 15,
    "structure": {...},
    "settings": {...}
}
```

Диалоговое окно экспорта

Класс `src/logiccraft/view/dialogs/project_export_dialog.py` предоставляет UI для:

- Выбора пути экспорта
- Настройки языка программирования
- Выбора архитектурного шаблона
- Включения/выключения опциональных компонентов
- Предпросмотра структуры проекта



Глава 9. Дополнительные UML-диаграммы

9.1. Use Case: актёры и сценарии

Помимо статической структуры системы, которую описывают *диаграммы классов*, важной частью анализа требований является поведенческое моделирование. Для этих целей в UML используются **диаграммы прецедентов (Use Case Diagrams)**. Они позволяют сфокусироваться на взаимодействии пользователей с системой и описать её функциональность с точки зрения конечного потребителя. В приложении *LogicCraft* реализована поддержка создания таких диаграмм, что позволяет проектировать не только кодовую базу, но и пользовательские сценарии в единой среде.

Ключевыми элементами диаграммы прецедентов являются **актёры (Actors)** и **прецеденты** или **сценарии (Use Cases)**.

- **Актёр** — это роль, которую играет кто-то или что-то вне системы, взаимодействуя с ней. Актёром может быть как человек (например, «Администратор», «Клиент»), так и другая система или аппаратное устройство. Визуально актёр изображается в виде стилизованной фигурки человека.
- **Сценарий (Прецедент)** — это последовательность действий, выполняемых системой, которая приносит актёру измеримый результат. По сути, это описание одной из функций системы с точки зрения пользователя.
- На диаграмме сценарий обычно рисуется в виде эллипса.

LogicCraft/src/logiccraft/models/diagram.py

Модели (данные)

- UseCaseActor — модель актёра (id, name, x, y)
- UseCaseScenario — модель сценария/use case (id, name, x, y, description)
- UseCaseConnection — связь между UC-элементами
- UMLDiagram содержит списки uc_actors, uc_scenarios, uc_connections
- ConnectionType включает UC-типы: uc_association, uc_include, uc_extend

class UseCaseActor(BaseModel):

"""Актёр Use Case диаграммы"""

id: str = Field(default_factory=lambda: str(uuid4()))

name: str

x: float

y: float

```
class UseCaseScenario(BaseModel):
    """Сценарий (Use Case) — овал"""
    id: str = Field(default_factory=lambda: str(uuid4()))
    name: str
    x: float
    y: float
    description: Optional[str] = None

class UseCaseConnection(BaseModel):
    """Связь между элементами Use Case диаграммы"""
    id: str = Field(default_factory=lambda: str(uuid4()))
    source_id: str
    target_id: str
    type: ConnectionType = ConnectionType.uc_association
    source_anchor: str = "right"
    target_anchor: str = "left"
```

Виджеты (отображение)

- ActorWidget — человечек (stick figure) с именем, поддерживает перемещение, rename, delete, якорные точки для связей (LogicCraft/src/logiccraft/view/widgets/actor_widget.py)

```
class ActorWidget(QGraphicsItem):
    """Актёр — стилизованный человечек с именем"""
    HEAD_R = 12
    BODY_H = 28
    ARM_W = 22
    LEG_H = 22
    FIGURE_H = HEAD_R * 2 + BODY_H + LEG_H
    FIGURE_W = ARM_W * 2

    def __init__(self, name: str, x: float = 0, y: float = 0,
                 actor_id: str = None):
        super().__init__()
        self.id = actor_id or str(uuid.uuid4())
        self.name = name
        self.setPos(x, y)
        self._is_selected = False
        self.signals = ActorSignals()
        self.setFlags(
            QGraphicsItem.GraphicsItemFlag.ItemIsMovable |
            QGraphicsItem.GraphicsItemFlag.ItemIsSelectable |
            QGraphicsItem.GraphicsItemFlag.ItemSendsGeometryChanges
        )
        self._label = QGraphicsTextItem(self)
        self._label.setPlainText(name)
        self._create_anchors()
```

```
def paint(self, painter: QPainter, option, widget=None):
    color = QColor("#7C3AED") if self._is_selected else QColor("#4B5563")
    pen = QPen(color, 2.0)
    painter.setPen(pen)
    # Голова, туловище, руки, ноги
    painter.drawEllipse(QPointF(0, self.HEAD_R), self.HEAD_R, self.HEAD_R)
    painter.drawLine(QPointF(0, self.HEAD_R*2), QPointF(0, self.HEAD_R*2+self.BODY_H))
    painter.drawLine(QPointF(-self.ARM_W, arm_y), QPointF(self.ARM_W, arm_y))
    painter.drawLine(QPointF(0, body_bottom), QPointF(-self.ARM_W*0.8, body_bottom+self.LEG_H))
    painter.drawLine(QPointF(0, body_bottom), QPointF(self.ARM_W*0.8, body_bottom+self.LEG_H))
```


- ScenarioWidget — овал с текстом внутри, аналогичный функционал (LogicCraft/src/logiccraft/view/widgets/scenario_widget.py)

```
class ScenarioWidget(QGraphicsItem):
    """Сценарий Use Case — овал с текстом внутри"""
```

```
DEFAULT_W = 160
DEFAULT_H = 60
```

```
def __init__(self, name: str, x: float = 0, y: float = 0,
             scenario_id: str = None, width=None, height=None):
    super().__init__()
    self.id = scenario_id or str(uuid.uuid4())
    self.name = name
    self.setPos(x, y)
    self._w = width or self.DEFAULT_W
    self._h = height or self.DEFAULT_H
    self.signals = ScenarioSignals()
    self._label = QGraphicsTextItem(self)
    self._label.setPlainText(name)
    self._create_anchors()
```

```
def paint(self, painter: QPainter, option, widget=None):
    # Тень
    painter.setPen(Qt.PenStyle.NoPen)
    painter.setBrush(QBrush(QColor(124, 58, 237, 20)))
    painter.drawEllipse(QRectF(2, 3, self._w, self._h))
    # Фон и рамка овала
    bg_color = QColor("#F3EEFF") if self._is_selected else QColor("#FFFFFF")
    border_color = QColor("#7C3AED") if self._is_selected else QColor("#9B72F5")
    painter.setPen(QPen(border_color, 2.0 if self._is_selected else 1.5))
    painter.setBrush(QBrush(bg_color))
    painter.drawEllipse(QRectF(0, 0, self._w, self._h))
```

Сцена и контроллер

- DiagramScene обрабатывает UC-виджеты наравне с UML-карточками: snap-to-grid, групповое выделение, показ якорных точек
- DiagramController имеет сигналы uc_actor_added/removed, uc_scenario_added/removed, uc_connection_added/removed

контролер (LogicCraft/src/logiccraft/controllers/diagram_controller.py)

```
class DiagramController(QObject):
    # Use Case сигналы
    uc_actor_added = pyqtSignal(object)    # UseCaseActor
    uc_actor_removed = pyqtSignal(str)
    uc_scenario_added = pyqtSignal(object) # UseCaseScenario
    uc_scenario_removed = pyqtSignal(str)
    uc_connection_added = pyqtSignal(object) # UseCaseConnection
    uc_connection_removed = pyqtSignal(str)
```

```
def add_uc_actor(self, x: float, y: float, name: str = None) -> Optional[UseCaseActor]:
    if name is None:
        name = f"Актёр[{len(self.manager.diagram.uc_actors) + 1}]"
    actor = UseCaseActor(name=name, x=x, y=y)
    self.manager.diagram.uc_actors.append(actor)
    self.uc_actor_added.emit(actor)
    self._save_state()
    return actor
```

```

def add_uc_scenario(self, x: float, y: float, name: str = None) -> Optional[UseCaseScenario]:
    if name is None:
        name = f"Сценарий{len(self.manager.diagram.uc_scenarios) + 1}"
    scenario = UseCaseScenario(name=name, x=x, y=y)
    self.manager.diagram.uc_scenarios.append(scenario)
    self.uc_scenario_added.emit(scenario)
    self._save_state()
    return scenario

```

Сцена : (LogicCraft/src/logiccraft/view/scenes/diagram_scene.py)

```

class DiagramScene(QGraphicsScene):
    connection_ready = pyqtSignal(str, str, str, str)
    card_moved = pyqtSignal(str, float, float)
    actor_renamed = pyqtSignal(str, str)
    scenario_renamed = pyqtSignal(str, str)

    def on_card_moved(self, card):
        """Snap to grid + групповое перемещение (UMLCard, ActorWidget, ScenarioWidget)"""
        from ..widgets.uml_card import UMLCard
        from ..widgets.actor_widget import ActorWidget
        from ..widgets.scenario_widget import ScenarioWidget
        grid = SceneStyle.GRID_STEP
        selected_items = [
            item for item in self.selectedItems()
            if isinstance(item, (UMLCard, ActorWidget, ScenarioWidget))
        ]
        targets = selected_items if len(selected_items) > 1 else [card]
        for c in targets:
            x = round(c.pos().x() / grid) * grid
            y = round(c.pos().y() / grid) * grid
            c.setPos(x, y)

```

Главное окно: (LogicCraft/src/logiccraft/view/main_window.py)

```

class MainWindow(QMainWindow):
    def __init__(self):
        self.uc_actor_map = {} # actor_id -> ActorWidget
        self.uc_scenario_map = {} # scenario_id -> ScenarioWidget
        self.uc_connection_map = {} # conn_id -> ConnectionLine

    def _on_uc_actor_added(self, actor_model):
        from ..widgets.actor_widget import ActorWidget
        widget = ActorWidget(
            name=actor_model.name, x=actor_model.x, y=actor_model.y,
            actor_id=actor_model.id
        )
        widget.signals.move_finished.connect(
            lambda aid, x, y: self.controller.on_uc_actor_move_finished(aid, x, y)
        )
        widget.signals.delete_requested.connect(self.controller.remove_uc_actor)
        self.scene.addItem(widget)
        self.uc_actor_map[actor_model.id] = widget

    def _on_uc_scenario_added(self, scenario_model):
        from ..widgets.scenario_widget import ScenarioWidget
        widget = ScenarioWidget(
            name=scenario_model.name, x=scenario_model.x, y=scenario_model.y,
            scenario_id=scenario_model.id
        )

```

```

)
widget.signals.move_finished.connect(
    lambda sid, x, y: self.controller.on_uc_scenario_move_finished(sid, x, y)
)
widget.signals.delete_requested.connect(self.controller.remove_uc_scenario)
self.scene.addItem(widget)
self.uc_scenario_map[scenario_model.id] = widget

```

9.2. Связи Include и Extend

Связь между актёром и сценарием называется **ассоциацией**. Она показывает, что данный актёр имеет право инициировать выполнение соответствующего сценария.

Более сложные поведенческие зависимости между самими сценариями описываются двумя специализированными типами связей:

Включение (<<include>>): указывает, что один сценарий (*базовый*) всегда обязательно включает в себя функциональность другого сценария (*включаемого*). Это используется для вынесения повторяющихся действий в отдельный блок. Например, сценарий «Оплатить заказ» может включать в себя обязательный сценарий «Авторизоваться в системе».

Расширение (<<extend>>): показывает, что один сценарий (*расширяющий*) добавляет свою функциональность к другому сценарию (*базовому*) при выполнении определённого условия. Это необязательное поведение. Например, сценарий «Применить промокод» может расширять сценарий «Оформить заказ», становясь доступным только если покупатель вводит действительный промокод.

Связи Include и Extend для Use Case диаграмм реализованы в следующих файлах:

Модель — LogicCraft/src/logiccraft/models/diagram.py

```

class ConnectionType(str, Enum):
    """Типы соединения"""
    # ...
    # Use Case связи
    uc_association = "uc_association" # актёр — сценарий
    uc_include = "uc_include"         # <<include>>
    uc_extend = "uc_extend"           # <<extend>>

```

Валидация — LogicCraft/src/logiccraft/controllers/diagram_controller.py

```

# Проверка корректности связей Include/Extend
for conn in diagram.uc_connections:
    if conn.type.value in ("uc_include", "uc_extend"):
        # Include/Extend должны связывать только сценарии
        if conn.source_id not in scenario_ids or conn.target_id not in scenario_ids:
            conn_type_name = "Include" if conn.type.value == "uc_include" else "Extend"
            warnings.append(f"✗ Связь {conn_type_name} должна связывать только сценарии")

```

Тулбокс (кнопки) — LogicCraft/src/logiccraft/view/panels/toolbox_panel.py

```

uc_conn_types = [
    ("—", "Ассоциация", "uc_association"),
    ("...→", "Include", "uc_include"),
    ("←...", "Extend", "uc_extend"),
]
for icon, label, ctype in uc_conn_types:
    btn = ToolboxButton(icon, label, ctype)
    btn.clicked.connect(lambda checked, t=ctype: self.set_connection_mode.emit(t))
    uc_conn_section.add_button(btn)

```

Визуализация линии — LogicCraft/src/logiccraft/view/widgets/connection_line.py

```

def _update_stereotype_label(self):
    """Обновляет стереотип-метку для Include/Extend"""
    type_value = self.connection_type.value if hasattr(self.connection_type, 'value') else str(self.connection_type)
    if type_value == "uc_include":
        self._stereotype_label.setPlainText("«include»")
        self._stereotype_label.setVisible(True)
    elif type_value == "uc_extend":
        self._stereotype_label.setPlainText("«extend»")
        self._stereotype_label.setVisible(True)
    else:
        self._stereotype_label.setPlainText("")
        self._stereotype_label.setVisible(False)

```

Пунктирная линия для Include/Extend задаётся здесь же:

```

if type_value in ("dependency", "realization", "uc_include", "uc_extend"):
    pen.setStyle(Qt.PenStyle.DashLine)

```

Наконечник стрелки — LogicCraft/src/logiccraft/view/widgets/arrow_head.py

```

elif type_value in ("uc_include", "uc_extend"):
    # Открытая стрелка, пунктирная линия (задаётся в connection_line)
    points = [QPointF(size, 0), QPointF(0, -size * 0.6), QPointF(size * 0.4, 0), QPointF(0, size * 0.6)]
    self.setPolygon(QPolygonF(points))
    self.setBrush(QBrush(Qt.BrushStyle.NoBrush))
    self.setPen(QPen(QColor(ArrowStyle.COLOR), ArrowStyle.WIDTH_NORMAL))

```

Определение типа связи при создании — LogicCraft/src/logiccraft/view/main_window.py

```

# Определяем, УС-связь или обычная
uc_types = {"uc_association", "uc_include", "uc_extend"}
all_uc_ids = set(self.uc_actor_map) | set(self.uc_scenario_map)
if source_id in all_uc_ids or target_id in all_uc_ids or conn_type in uc_types:
    # Создаём УС-связь

```

9.3. Переключение между типами диаграмм

В LogicCraft реализована возможность работы с разными типами UML-диаграмм (например, «Диаграмма классов», «Диаграмма прецедентов») в рамках одного проекта.

Переключение между ними происходит через селектор в интерфейсе. При смене типа диаграммы:

Визуальное представление: на холсте отображаются только элементы, относящиеся к выбранному типу.

Панель инструментов: набор доступных для добавления элементов меняется в соответствии с активным типом диаграммы.

Все элементы сохраняются в единой модели данных UMLDiagram, что обеспечивает их целостность и позволяет бесшовно возвращаться к ранее созданным диаграммам. [sources=[file]]

Модель — LogicCraft/src/logiccraft/models/diagram.py

```

class DiagramType(str, Enum):
    """Тип диаграммы"""
    CLASS = "class"
    USE_CASE = "use_case"

```

```

class UMLDiagram(BaseModel):
    """Модель диаграммы"""

```

```

id: str = Field(default_factory=lambda: str(uuid4()))
name: str
diagram_type: DiagramType = DiagramType.CLASS
nodes: list[UMLNode] = Field(default_factory=list)
connections: list[UMLConnection] = Field(default_factory=list)
# Use Case элементы
uc_actors: List["UseCaseActor"] = Field(default_factory=list)
uc_scenarios: List["UseCaseScenario"] = Field(default_factory=list)
uc_connections: List["UseCaseConnection"] = Field(default_factory=list)

```

Поле `diagram_type` определяет, какой набор элементов активен. По умолчанию — `DiagramType.CLASS`.

UI: комбобокс выбора типа — `LogicCraft/src/logiccraft/view/main_window.py`

```

self.diagram_type_combo = QComboBox()
self.diagram_type_combo.addItem(" Диаграмма классов", "class")
self.diagram_type_combo.addItem(" Use Case диаграмма", "use_case")
self.diagram_type_combo.setCurrentIndex(0)
self.diagram_type_combo.currentIndexChanged.connect(self._on_diagram_type_changed)

```

При изменении индекса вызывается `_on_diagram_type_changed`, который:

Обновляет `diagram_type` в модели

Переключает видимые секции в тулбоксе

Показывает/скрывает соответствующие элементы на холсте

Обработчик переключения — `LogicCraft/src/logiccraft/view/main_window.py`

```

def _on_diagram_type_changed(self, index: int):
    """Обработка переключения типа диаграммы"""
    diagram_type = self.diagram_type_combo.itemData(index)

    # Обновляем тип диаграммы в модели
    self.controller.manager.diagram.diagram_type = diagram_type

    # Обновляем панель инструментов - показываем только нужные секции
    self.toolbox_panel.set_diagram_type(diagram_type)

    # Обновляем видимость элементов в зависимости от типа
    if diagram_type == "class":
        # Показываем классы, скрываем Use Case элементы
        for card in self.card_map.values():
            card.setVisible(True)
        for conn in self.connection_map.values():
            conn.setVisible(True)
        for actor in self.uc_actor_map.values():
            actor.setVisible(False)
        for scenario in self.uc_scenario_map.values():
            scenario.setVisible(False)
        for uc_conn in self.uc_connection_map.values():
            uc_conn.setVisible(False)
        self.update_status("Переключено на диаграмму классов")
    else: # use_case
        # Скрываем классы, показываем Use Case элементы
        for card in self.card_map.values():
            card.setVisible(False)
        for conn in self.connection_map.values():
            conn.setVisible(False)

```

```

for actor in self.uc_actor_map.values():
    actor.setVisible(True)
for scenario in self.uc_scenario_map.values():
    scenario.setVisible(True)
for uc_conn in self.uc_connection_map.values():
    uc_conn.setVisible(True)
self.update_status("Переключено на Use Case диаграммы")

```

Тулбок — LogicCraft/src/logiccraft/view/panels/toolbox_panel.py

```

def set_diagram_type(self, diagram_type: str):
    """Показать только инструменты для выбранного типа диаграммы"""
    # Получаем все секции
    sections = self.findChildren(ToolboxSection)

    for section in sections:
        title = section.title() if hasattr(section, 'title') else ""
        # Получаем заголовок из первого QLabel
        label = section.findChild(QLabel)
        section_title = label.text() if label else ""

        if diagram_type == "class":
            # Показываем только секции для диаграммы классов
            if section_title in ["Классы", "Связи классов"]:
                section.setVisible(True)
            else:
                section.setVisible(False)
        elif diagram_type == "use_case":
            # Показываем только секции для Use Case диаграммы
            if section_title in ["Use Case", "Связи Use Case"]:
                section.setVisible(True)
            else:
                section.setVisible(False)

```

Тулбок содержит 4 секции: «Классы», «Связи классов», «Use Case», «Связи Use Case». При переключении типа диаграммы показываются только релевантные секции.

Создание нового проекта — LogicCraft/src/logiccraft/main.py

```

def _on_welcome_new_project(self):
    """Обработка создания нового проекта"""
    if hasattr(self.welcome_dialog, 'last_project_config'):
        self.project_config = self.welcome_dialog.last_project_config

    # Устанавливаем тип диаграммы
    diagram_type = self.project_config.get("diagram_type", "class")
    self.controller.manager.diagram.diagram_type = diagram_type

    project_name = self.project_config.get("name", "Untitled")
    self.window.setWindowTitle(f"LogicCraft — {project_name}")

    self._create_project_structure()
    # Обновляем панель инструментов для выбранного типа диаграммы
    self.window.toolbox_panel.set_diagram_type(diagram_type)
    # Синхронизируем комбобокс
    for i in range(self.window.diagram_type_combo.count()):
        if self.window.diagram_type_combo.itemData(i) == diagram_type:

```

```
self.window.diagram_type_combo.setCurrentIndex(i)
break
```

```
self.window.show()
```

Загрузка диаграммы — LogicCraft/src/logiccraft/view/main_window.py

```
"""Обработка загрузки диаграммы - обновляем панель инструментов"""
diagram_type = self.controller.manager.diagram.diagram_type
# Синхронизируем комбобокс с типом загруженной диаграммы
for i in range(self.diagram_type_combo.count()):
    if self.diagram_type_combo.itemData(i) == diagram_type:
        self.diagram_type_combo.setCurrentIndex(i)
        break
# Обновляем панель инструментов
self.toolbox_panel.set_diagram_type(diagram_type)
```

Переключение типа диаграммы затрагивает три слоя:

Модель — поле `diagram_type` в `UMLDiagram`

Контроллер — не имеет отдельного метода, тип меняется напрямую в модели

View — комбобокс в тулбаре, секции тулбокса, видимость элементов на сцене

При переключении элементы не удаляются, а только скрываются/показываются, что позволяет сохранять данные обоих типов в одном файле проекта.

9.4. Визуализация элементов разных диаграмм

В LogicCraft можно одновременно работать с элементами разных UML-диаграмм (например, классов и прецедентов) на одном холсте.

Это достигается за счёт:

Единой модели данных `UMLDiagram`, где хранятся все элементы.

Фильтрации: при выборе типа диаграммы отображаются только релевантные ей элементы.

Визуального различия: разные типы элементов (классы, актёры) имеют свой уникальный стиль.

Это позволяет создавать комплексные схемы, например, показывать, как пользователь (актёр) взаимодействует с конкретным классом, объединяя структурный и поведенческий анализ. `[sources=[file]]`.

LogicCraft использует единую сцену (`DiagramScene`) для обоих типов диаграмм. При переключении типа элементы не удаляются, а только меняется их видимость и доступные инструменты. Визуализация каждого элемента определяется его виджетом.

UML-карточка (классы) — `LogicCraft/src/logiccraft/view/widgets/uml_card.py`

```
class UMLCard(QGraphicsRectItem):
    """Карточка класса UML с поддержкой drag & drop"""
    HEADER_HEIGHT = 36
    RADIUS = 12

    def paint(self, painter: QPainter, option, widget=None):
        """Кастомная отрисовка с скруглёнными углами"""
        r = self.rect()
        radius = self.RADIUS
        h = self.HEADER_HEIGHT

        painter.setRenderHint(QPainter.RenderHint.Antialiasing)

        # Тень
        shadow_color = QColor(124, 58, 237, 25)
        painter.setPen(Qt.PenStyle.NoPen)
        painter.setBrush(QBrush(shadow_color))
```

```
painter.drawRoundedRect(r.adjusted(2, 3, 2, 3), radius, radius)
```

```
# Основной фон карточки (белый)
```

```
border_color = QColor(CardStyle.SELECTED_BORDER) if self._is_selected else QColor(CardStyle.BORDER)
border_width = CardStyle.SELECTED_BORDER_WIDTH if self._is_selected else CardStyle.BORDER_WIDTH
painter.setPen(QPen(border_color, border_width))
painter.setBrush(QBrush(QColor(CardStyle.BACKGROUND)))
painter.drawRoundedRect(r, radius, radius)
```

```
# Определяем цвет заголовка по типу узла
```

```
node_type = getattr(self, 'node_type', 'class')
```

```
if hasattr(node_type, 'value'):
```

```
    node_type = node_type.value
```

```
if node_type == 'interface':
```

```
    header_color = QColor("#9B72F5") # Светло-фиолетовый для интерфейсов
```

```
elif node_type == 'enum':
```

```
    header_color = QColor("#10B981") # Зелёный для enum
```

```
elif node_type == 'abstract_class':
```

```
    header_color = QColor("#8B5CF6") # Средне-фиолетовый для абстрактных
```

```
else:
```

```
    header_color = QColor(CardStyle.HEADER_BG) # Стандартный фиолетовый
```

```
# Заголовок — цветной прямоугольник с скруглением только сверху
```

```
header_path = QPainterPath()
```

```
header_path.moveTo(r.left() + radius, r.top())
```

```
header_path.lineTo(r.right() - radius, r.top())
```

```
header_path.arcTo(r.right() - radius * 2, r.top(), radius * 2, radius * 2, 90, -90)
```

```
header_path.lineTo(r.right(), r.top() + h)
```

```
header_path.lineTo(r.left(), r.top() + h)
```

```
header_path.arcTo(r.left(), r.top(), radius * 2, radius * 2, 180, -90)
```

```
header_path.closeSubpath()
```

```
painter.setPen(Qt.PenStyle.NoPen)
```

```
painter.setBrush(QBrush(header_color))
```

```
painter.drawPath(header_path)
```

Заголовок карточки меняет цвет в зависимости от типа узла:

Class — стандартный фиолетовый #7C3AED

Abstract Class — средне-фиолетовый #8B5CF6

Interface — светло-фиолетовый #9B72F5

Enum — зелёный #10B981

Также добавляется стереотип и иконка:

```
def update_content(self):
```

```
    # Визуальное отличие по типу
```

```
    if node_type == 'interface':
```

```
        stereotype = '«interface»'
```

```
        icon = '◇'
```

```
        name_style = 'font-style:italic; font-weight:bold;'
```

```
    elif node_type == 'enum':
```

```
        stereotype = '«enumeration»'
```

```
        icon = '≡'
```

```
        name_style = 'font-weight:bold;'
```



```

elif node_type == 'abstract_class':
    stereotype = '«abstract»'
    icon = '△'
    name_style = 'font-style:italic; font-weight:bold;'
else:
    stereotype = ''
    icon = '◯'
    name_style = 'font-weight:bold;'

# Формируем HTML заголовка
if stereotype:
    header_html = (
        f'<div style="text-align:center;">'
        f'<div style="font-size:9px; color:rgba(255,255,255,200); margin-bottom:2px;">'
        f'{stereotype}</div>'
        f'<div style="font-size:13px; {name_style}">'
        f'{icon} {self.name}</div>'
        f'</div>'
    )
else:
    header_html = (
        f'<div style="text-align:center; font-size:13px; {name_style}">'
        f'{icon} {self.name}</div>'
    )

```

Актёр (Use Case) — LogicCraft/src/logiccraft/view/widgets/actor_widget.py

```

class ActorWidget(QGraphicsItem):
    """Актёр — стилизованный человек с именем"""
    HEAD_R = 12    # радиус головы
    BODY_H = 28    # высота туловища
    ARM_W = 22     # полуширина рук
    LEG_H = 22     # высота ног
    FIGURE_H = HEAD_R * 2 + BODY_H + LEG_H
    FIGURE_W = ARM_W * 2

    def paint(self, painter: QPainter, option, widget=None):
        painter.setRenderHint(QPainter.RenderHint.Antialiasing)

        color = QColor("#7C3AED") if self._is_selected else QColor("#4B5563")
        pen = QPen(color, 2.0)
        painter.setPen(pen)
        painter.setBrush(QBrush(Qt.BrushStyle.NoBrush))

        cx = 0.0
        top = 0.0

        # Голова
        head_cx = cx
        head_cy = top + self.HEAD_R
        painter.drawEllipse(QPointF(head_cx, head_cy), self.HEAD_R, self.HEAD_R)

        # Туловище
        body_top = top + self.HEAD_R * 2

```

```

body_bottom = body_top + self.BODY_H
painter.drawLine(QPointF(cx, body_top), QPointF(cx, body_bottom))

# Руки
arm_y = body_top + self.BODY_H * 0.35
painter.drawLine(QPointF(cx - self.ARM_W, arm_y), QPointF(cx + self.ARM_W, arm_y))

# Ноги
painter.drawLine(QPointF(cx, body_bottom), QPointF(cx - self.ARM_W * 0.8, body_bottom + self.LEG_H))
painter.drawLine(QPointF(cx, body_bottom), QPointF(cx + self.ARM_W * 0.8, body_bottom + self.LEG_H))

# Выделение — пунктирный прямоугольник
if self._is_selected:
    sel_pen = QPen(QColor("#7C3AED"), 1.5, Qt.PenStyle.DashLine)
    painter.setPen(sel_pen)
    painter.setBrush(QBrush(Qt.BrushStyle.NoBrush))
    painter.drawRoundedRect(self.boundingRect().adjusted(2, 2, -2, -2), 6, 6)

```

Актёр отрисовывается как stick figure (человечек) из круга (голова), линии (туловище), горизонтальной линии (руки) и двух линий (ноги). Имя отображается под фигурой.

Сценарий (Use Case) — LogicCraft/src/logiccraft/view/widgets/scenario_widget.py

```

class ScenarioWidget(QGraphicsItem):
    """Сценарий Use Case — овал с текстом внутри"""
    DEFAULT_W = 160
    DEFAULT_H = 60

    def paint(self, painter: QPainter, option, widget=None):
        painter.setRenderHint(QPainter.RenderHint.Antialiasing)

        # Тень
        shadow_color = QColor(124, 58, 237, 20)
        painter.setPen(Qt.PenStyle.NoPen)
        painter.setBrush(QBrush(shadow_color))
        painter.drawEllipse(QRectF(2, 3, self._w, self._h))

        # Фон овала
        bg_color = QColor("#F3EEFF") if self._is_selected else QColor("#FFFFFF")
        border_color = QColor("#7C3AED") if self._is_selected else QColor("#9B72F5")
        border_w = 2.0 if self._is_selected else 1.5

        painter.setPen(QPen(border_color, border_w))
        painter.setBrush(QBrush(bg_color))
        painter.drawEllipse(QRectF(0, 0, self._w, self._h))

```

Сценарий — это овал с тенью, белым фоном и фиолетовой рамкой. Текст имени центрируется внутри овала.

Линии связи — LogicCraft/src/logiccraft/view/widgets/connection_line.py

```

class ConnectionLine(QGraphicsLineItem):
    """Линия связи с защитой от дублирования наконечников"""

    def __init__(self, source, target,
                 source_anchor: str = "right", target_anchor: str = "left",
                 connection_type: ConnectionType = ConnectionType.ASSOCIATION,
                 connection_id: str = None,

```

```

multiplicity: str = None,
name: str = None):
# Настройка пера
pen = QPen(QColor(ConnectionStyle.LINE_COLOR), ConnectionStyle.LINE_WIDTH)
type_value = connection_type.value if hasattr(connection_type, 'value') else str(connection_type)
if type_value in ("dependency", "realization", "uc_include", "uc_extend"):
    pen.setStyle(Qt.PenStyle.DashLine) # Пунктир для зависимостей и UC-связей
self.setPen(pen)

# Наконечник
self.arrow_head = ArrowHead(QPointF(1, 0), self.connection_type)

# Стереотип-метка для Include/Extend
self._stereotype_label = QGraphicsTextItem("", self)
self._stereotype_label.setFont(QFont("Inter", 8, QFont.Weight.Medium))
self._stereotype_label.setDefaultTextColor(QColor("#7C3AED"))
self._update_stereotype_label()

```

Для Include/Extend линия становится пунктирной и добавляется текстовая метка «include» или «extend»:

```

def _update_stereotype_label(self):
    """Обновляет стереотип-метку для Include/Extend"""
    type_value = self.connection_type.value if hasattr(self.connection_type, 'value') else str(self.connection_type)
    if type_value == "uc_include":
        self._stereotype_label.setPlainText("«include»")
        self._stereotype_label.setVisible(True)
    elif type_value == "uc_extend":
        self._stereotype_label.setPlainText("«extend»")
        self._stereotype_label.setVisible(True)
    else:
        self._stereotype_label.setPlainText("")
        self._stereotype_label.setVisible(False)

```

Наконечник стрелки — LogicCraft/src/logiccraft/view/widgets/arrow_head.py

```

def _update_shape(self):
    """Обновляет форму наконечника в зависимости от типа связи"""
    size = ArrowStyle.SIZE
    type_value = self.connection_type.value if hasattr(self.connection_type, 'value') else str(self.connection_type)

    if type_value == "inheritance":
        # Пустой треугольник
        points = [QPointF(size, 0), QPointF(0, -size * 0.6), QPointF(0, size * 0.6)]
        self.setPolygon(QPolygonF(points))
        self.setBrush(QBrush(Qt.BrushStyle.NoBrush))

    elif type_value == "composition":
        # Закрашенный ромб
        points = [QPointF(size, 0), QPointF(size / 2, -size * 0.6), QPointF(0, 0), QPointF(size / 2, size * 0.6)]
        self.setPolygon(QPolygonF(points))
        self.setBrush(QBrush(QColor(ArrowStyle.COLOR)))

    elif type_value in ("uc_include", "uc_extend"):
        # Открытая стрелка (как dependency)
        points = [QPointF(size, 0), QPointF(0, -size * 0.6), QPointF(size * 0.4, 0), QPointF(0, size * 0.6)]
        self.setPolygon(QPolygonF(points))

```

```
self.setBrush(QBrush(Qt.BrushStyle.NoBrush))
```

```
elif type_value == "uc_association":
```

```
# Простая линия без наконечника
```

```
points = [QPointF(0, 0)]
```

```
self.setPolygon(QPolygonF(points))
```

```
self.setPen(QPen(Qt.PenStyle.NoPen))
```

```
else: # association, interaction
```

```
# Закрашенная стрелка
```

```
points = [QPointF(size, 0), QPointF(0, -size * 0.6), QPointF(0, size * 0.6)]
```

```
self.setPolygon(QPolygonF(points))
```

```
self.setBrush(QBrush(QColor(ArrowStyle.COLOR)))
```

Переключение видимости — LogicCraft/src/logiccraft/view/main_window.py

```
def _on_diagram_type_changed(self, index: int):
```

```
    diagram_type = self.diagram_type_combo.itemData(index)
```

```
    self.controller.manager.diagram.diagram_type = diagram_type
```

```
    self.toolbox_panel.set_diagram_type(diagram_type)
```

```
if diagram_type == "class":
```

```
# Показываем классы, скрываем Use Case элементы
```

```
for card in self.card_map.values():
```

```
    card.setVisible(True)
```

```
for conn in self.connection_map.values():
```

```
    conn.setVisible(True)
```

```
for actor in self.uc_actor_map.values():
```

```
    actor.setVisible(False)
```

```
for scenario in self.uc_scenario_map.values():
```

```
    scenario.setVisible(False)
```

```
for uc_conn in self.uc_connection_map.values():
```

```
    uc_conn.setVisible(False)
```

```
self.update_status("Переключено на диаграмму классов")
```

```
else: # use_case
```

```
# Скрываем классы, показываем Use Case элементы
```

```
for card in self.card_map.values():
```

```
    card.setVisible(False)
```

```
for conn in self.connection_map.values():
```

```
    conn.setVisible(False)
```

```
for actor in self.uc_actor_map.values():
```

```
    actor.setVisible(True)
```

```
for scenario in self.uc_scenario_map.values():
```

```
    scenario.setVisible(True)
```

```
for uc_conn in self.uc_connection_map.values():
```

```
    uc_conn.setVisible(True)
```

```
self.update_status("Переключено на Use Case диаграмму")
```

Все элементы хранятся в словарях MainWindow (card_map, uc_actor_map, uc_scenario_map, connection_map, uc_connection_map) и переключаются видимостью без удаления из сцены.

10.1. Unit-тесты для моделей

Модели данных — это основа нашего приложения. Они отвечают за хранение и валидацию UML-элементов: классов, интерфейсов, связей и диаграмм. Чтобы гарантировать их надежность, мы создали обширный набор unit-тестов. Они расположены в файле `LogicCraft/test/test_models.py`. Их цель — покрыть каждую модель и исключить риск появления багов при рефакторинге.

Что именно покрывают тесты?

ниже представлены часть кода, полную версию можно посмотреть на GitHub(<https://github.com/Trewasasha/LogicCraft>)

UMLProperty и UMLMethod:

Проверяется создание свойств и методов с различными настройками видимости, статичностью и значениями по умолчанию. Отдельно тестируется создание абстрактных методов.

`class TestUMLProperty:`

```
"""Tests for UMLProperty model."""
```

```
def test_create_property(self):
```

```
    """Test creating a basic property."""
```

```
    prop = UMLProperty(name="username", type="str")
```

```
    assert prop.name == "username"
```

```
    assert prop.type == "str"
```

```
    assert prop.visibility == "public" # default
```

```
    assert prop.is_static is False
```

```
    assert prop.default_value is None
```

UMLNode:

Здесь фокус на проверке автоматической генерации UUID, добавлении свойств и методов, а также на корректной работе стереотипов («interface», «enum», «abstract»).

`class TestUMLNode:`

```
"""Tests for UMLNode model."""
```

```
def test_create_node(self):
```

```
    """Test creating a node with auto-generated ID."""
```

```
    node = UMLNode(name="User", x=100.0, y=200.0)
```

```
    assert node.name == "User"
```

```
    assert node.x == 100.0
```

```
    assert node.y == 200.0
```

```
    assert isinstance(node.id, str)
```

```
    # Check it's a valid UUID
```

```
    UUID(node.id)
```

UMLConnection:

Тестируется создание связей разного типа: ассоциации, наследования, композиции и зависимостей. Особенное внимание уделяется связям Use Case («include», «extend») и правильной передаче анкерных точек.

`class TestUMLConnection:`

```
"""Tests for UMLConnection model."""
```

```
def test_create_connection(self):
```

```
    """Test creating a connection."""
```

```
    conn = UMLConnection(
```

```
        source_id="node-1",
```

```
        target_id="node-2",
```

```
        type=ConnectionType.association
```

```
    )
```

```

assert conn.source_id == "node-1"
assert conn.target_id == "node-2"
assert conn.type == ConnectionType.association
assert isinstance(conn.id, str)

```

UMLDiagram:

Самый важный тест — сериализация и десериализация. Мы проверяем, что диаграмма сохраняется в JSON и восстанавливается без потери данных.

class TestUMLDiagram:

```

"""Tests for UMLDiagram model."""

```

```

def test_create_diagram(self):
    """Test creating a diagram."""
    diagram = UMLDiagram(
        name="Shop System",
        nodes=[],
        connections=[]
    )

    assert diagram.name == "Shop System"
    assert diagram.nodes == []
    assert diagram.connections == []

```

10.2. Тесты генерации кода

В LogicCraft генерация кода — это сложный процесс, который охватывает несколько шагов: от рендеринга шаблонов до записи файлов на диск.

Где лежат тесты?

В файле [LogicCraft/test/test_codegen.py](#).

Зачем они нужны?

Чтобы удостовериться, что логика генерации работает корректно, а реальные операции с диском проходят гладко, ниже представлены часть кода, полную версию можно посмотреть на GitHub(<https://github.com/Trewasasha/LogicCraft>)

Какие этапы покрыты?

Шаблонный рендеринг: проверяется, что Jinja2-шаблоны правильно подставляют данные из UML-диаграммы.

class TestJinja2Templates:

```

"""Tests for Jinja2 template setup (Week 1, Month 4)."""

```

```

def test_template_class_generation(self):
    """Test Jinja2 template generates Python class."""
    template = TemplateEngine()

    class_data = {
        "name": "User",
        "properties": [
            {"name": "username", "type": "str", "visibility": "public"},
            {"name": "_password", "type": "str", "visibility": "private"}
        ],
        "methods": [
            {"name": "get_username", "return_type": "str", "visibility": "public", "parameters": []}
        ]
    }

    result = template.render_class(class_data)

```

```

assert "class User:" in result
assert "def __init__(self)" in result
assert "self.username" in result
assert "self._password" in result
assert "def get_username(self)" in result

```

Перевод связей: проверяется, как UML-связи (наследование, композиция, ассоциации) отражаются в коде.

class TestConnectionTranslation:

```

def test_inheritance_to_class_parent(self):
    """Test inheritance connection translates to parent class."""
    translator = ConnectionTranslator()

    classes = [
        {"id": "c1", "name": "User", "properties": [], "methods": []},
        {"id": "c2", "name": "Admin", "properties": [], "methods": []}
    ]
    connections = [
        {"source_id": "c1", "target_id": "c2", "type": "inheritance"}
    ]

    result = translator.translate(classes, connections)

    assert result["c2"]["parent"] == "User"

```

Предпросмотр: тестируется, как открывается окно с вкладками для каждого файла.

class TestCodePreview:

```

"""Tests for code preview window (Week 3, Month 4)."""

def test_preview_shows_generated_files(self):
    """Test preview window shows list of generated files."""
    preview = CodePreviewWindow()

    files = {
        "user.py": "class User:...",
        "order.py": "class Order:..."
    }

    preview.load_files(files)

    assert len(preview.tabs) == 2
    assert "user.py" in preview.tabs
    assert "order.py" in preview.tabs

```

Экспорт на диск: проверяется создание папок и файлов.

class TestProjectExport:

```

"""Tests for project export to disk (Week 4, Month 4)."""

def test_export_creates_directory_structure(self):
    """Test export creates proper directory structure."""
    exporter = ProjectExporter()

    project = {
        "name": "MyProject",
        "files": {
            "models/user.py": "class User:...",
            "models/order.py": "class Order:...",
            "main.py": "from models.user import User"
        }
    }

```

```
}
}
```

Почему тесты идут быстро?

Потому что мы используем **плейсхолдеры** — специальные заглушки, которые имитируют работу [CodeGenerator](#). Это позволяет тестировать логику, не дожидаясь реальных операций с файловой системой. [sources=file]

Placeholder classes and functions

class TemplateEngine:

"""Placeholder: Jinja2 template engine for code generation."""

def render_class(self, class_data):

lines = []

Imports for abstract classes

if class_data.get("is_abstract"):

lines.append("from abc import ABC, abstractmethod")

Class definition

parent = class_data.get("parent", "")

if class_data.get("is_abstract"):

parent = "ABC" **if not** parent **else** f"{parent}, ABC"

parent_str = f"({parent})" **if** parent **else** ""

lines.append(f"class {class_data['name']}{parent_str}:")

__init__ method

lines.append(" def __init__(self):")

for prop in class_data.get("properties", []):

default = f" = {prop['default_value']}" **if** prop.get("default_value") **else** ""

lines.append(f" self.{prop['name']}{default}")

if not class_data.get("properties"):

lines.append(" pass")

Methods

for method in class_data.get("methods", []):

if method.get("is_abstract"):

lines.append(" @abstractmethod")

if method.get("is_static"):

lines.append(" @staticmethod")

params_str = ", ".join([f"{p['name']}: {p['type']}" **for** p in method.get("parameters", [])])

self_param = "self" **if not** method.get("is_static") **else** ""

if self_param **and** params_str:

params_str = f"{self_param}, {params_str}"

elif self_param:

params_str = self_param

lines.append(f" def {method['name']}{params_str} -> {method['return_type']}:")

lines.append(" pass")

return "\n".join(lines)

В LogicCraft тесты генерации кода охватывают все этапы: от рендеринга шаблонов до записи файлов на диск.

Их цель — защитить ядро приложения от регрессий. Мы не тестируем реальное создание файлов, а проверяем логику с помощью заглушек. Это ускоряет тесты и снижает риски сбоев.

10.3. Тестирование графического интерфейса

Все тесты для графического интерфейса находятся в папке [LogicCraft/test](#).

ниже представлены часть кода, полную версию можно посмотреть на GitHub(<https://github.com/Trewasasha/LogicCraft>)

- Стили и оформление: [test_view_styles.py](#) .
Здесь проверяется, что все компоненты используют только цвета из [theme.py](#). Есть smoke-тесты на `hardcode` и гипотезы (hypothesis) для `UMLCard`, `ArrowHead` и других.

`class` `TestThemeStructure`:

```
"""Requirement 1: Централизованное хранилище Style-Token"""
```

```
def test_theme_has_all_sections(self):
```

```
    """1.1, 1.2, 8.1 — все секции присутствуют"""
```

```
    from logiccraft.view.theme import (
```

```
        CardStyle, ConnectionStyle, SceneStyle, AnchorStyle, ArrowStyle
```

```
)
```

```
    assert CardStyle is not None
```

```
    assert ConnectionStyle is not None
```

```
    assert SceneStyle is not None
```

```
    assert AnchorStyle is not None
```

```
    assert ArrowStyle is not None
```

```
def test_theme_tokens_accessible_without_init(self):
```

```
    """1.4 — токены доступны сразу после импорта"""
```

```
    from logiccraft.view.theme import CardStyle
```

```
    # Не должно бросать исключений
```

```
    _ = CardStyle.BACKGROUND
```

```
    _ = CardStyle.BORDER
```

- UX и взаимодействие: [test_editor_ux.py](#) .

Здесь тестируются `InspectorPanel`, горячие клавиши, контекстное меню и история. Используется библиотека `hypothesis`.

`class` `TestInspectorPanel`:

```
"""Tests for inspector panel (Week 1, Month 3)."""
```

```
def test_inspector_panel_shows_selected_class(self):
```

```
    """Test inspector panel displays properties of selected class."""
```

```
    selected_class = {
```

```
        "id": "class_1",
```

```
        "name": "User",
```

```
        "properties": [
```

```
            {"name": "username", "type": "str", "visibility": "public"}
```

```
        ],
```

```
        "methods": [
```

```
            {"name": "get_username", "return_type": "str", "visibility": "public"}
```

```
        ]
```

```
    }
```

```
    inspector = InspectorPanel()
```

```
    inspector.load_class(selected_class)
```

```
    assert inspector.current_class == selected_class
```

```
    assert inspector.name_field.value == "User"
```

```
    assert len(inspector.properties_list) == 1
```

```
    assert len(inspector.methods_list) == 1
```

10.4. Запуск тестов через pytest

В LogicCraft мы выбрали pytest как основной инструмент для автоматического тестирования. Он отлично подходит для больших проектов, так как позволяет комбинировать разные подходы.

Как запустить тесты?

Быстрая проверка: `$ pytest -k <имя теста>`. Например, `pytest -k test_theme`.

Полный запуск: `$ pytest`. Проверяются все тесты в папке `./tests`.

Выборочная группа: `$ pytest ./tests/test_models.py`.

Чем хорош pytest?

Простота организации: тесты живут рядом с кодом, в папке `/tests`.

Поддержка плагины: мы используем `pytest-hypothesis` для генерации тысяч случаев.

Интеграция с CI: тесты запускаются в GitHub Actions при каждом пуш- или пулл-запросе.

Наши лучшие практики

Плейсхолдеры: мы заменяем тяжелые зависимости (например, PyQt) на легкие заглушки. Это ускоряет тесты в десятки раз.

Гипотезы: вместо ручного подбора данных мы доверяем это библиотеке `hypothesis`. Она находит краевые случаи, которые сложно предусмотреть.

Благодаря этому мы уверены, что новая функция не ломает старую, а рефакторинг безопасен.

Глава 11. Сборка и развёртывание

Эта глава посвящена тому, как собрать LogicCraft в автономное приложение для Windows и macOS.

11.1. Сборка исполняемого файла с PyInstaller

Для упаковки Python-приложения в EXE-файл мы используем PyInstaller. Это удобно: программа запускается одним щелчком, и пользователю не нужен Python.

Файл спецификации: `logiccraft.spec` (в корне проекта).

ниже представлены часть кода, полную версию можно посмотреть на GitHub(<https://github.com/Trewasasha/LogicCraft>)

```
import os
```

```
from pathlib import Path
```

```
block_cipher = None
```

```
project_root = os.path.abspath(os.path.dirname(SPEC))
```

```
a = Analysis(
```

```
    [os.path.join(project_root, 'src', 'logiccraft', '__main__.py')],
```

```
    pathex=[os.path.join(project_root, 'src')],
```

```
    binaries=[],
```

```
    datas=[
```

```
        # Resources (icons)
```

```
        (os.path.join(project_root, 'resources', 'icons'), 'resources/icons'),
```

```
        # Jinja2 templates
```

```
        (os.path.join(project_root, 'src', 'logiccraft', 'templates', '*.j2'), 'logiccraft/templates'),
```

```
        (os.path.join(project_root, 'src', 'logiccraft', 'templates', 'config.py'), 'logiccraft/templates'),
```

```
        (os.path.join(project_root, 'src', 'logiccraft', 'templates', '__init__.py'), 'logiccraft/templates'),
```

```
        # QSS stylesheet
```

```
        (os.path.join(project_root, 'src', 'logiccraft', 'style.qss'), 'logiccraft'),
```

```
    ],
```

Он собирает все необходимые ресурсы:

- **Иконка:** `resources/icons/icon2.png`.
- **Шрифты:** `*.ttf` и `*.otf`.
- **Шаблоны:** `src/logiccraft/templates/*.j2`.
- **CSS:** `src/logiccraft/style.qss`.

Команды для сборки:

```
bashКопировать
```

```
pyinstaller logiccraft.spec # Базовая команда
```

pyinstaller --clean --noconfirm logiccraft.спес # Удаляет временные файлы

Настройки сборки:

- Входная точка: src/logiccraft/__main__.py.
- Режим: console=False — приложение без консольного окна.
- Упаковщик: UPX (сжимает размер файла).

После сборки получаем готовую программу в папке dist/LogicCraft/

11.2. Запуск на Windows, macOS

Поскольку проект написан на Python и использует библиотеку PyQt6, которая сама по себе является кроссплатформенной, итоговое приложение работает нативно на обеих операционных системах без необходимости установки интерпретатора Python у конечного пользователя.

Процесс запуска выглядит следующим образом:

1. Windows (.exe):

- В папке dist/LogicCraft создается файл LogicCraft.exe.
- Для работы приложения достаточно просто запустить этот .exe-файл двойным щелчком мыши.
- Все необходимые библиотеки (DLL) и ресурсы уже включены в эту директорию или находятся внутри одного исполняемого файла (в зависимости от настроек сборки).

2. macOS (.app bundle):

- PyInstaller создает стандартное для macOS приложение — пакет (bundle) с расширением .app (например, LogicCraft.app).
- Пользователь может перенести этот пакет в папку «Программы» или запустить его прямо из каталога сборки.
- Приложение будет работать как обычное нативное приложение macOS.

Таким образом, LogicCraft обеспечивает одинаковый пользовательский опыт независимо от того, используете вы ПК на Windows или ноутбук Apple.

Заключение

LogicCraft — это наш успех. Мы создали мощный, кроссплатформенный инструмент, который автоматизирует разработку от UML-диаграммы до готового кода на Python и Java.

Мы гордимся результатом: стабильная архитектура, интуитивный интерфейс и умная генерация кода экономят время разработчиков и повышают качество их работы. **Миссия выполнена!**

